

尚硅谷大数据项目之 Paimon 湖仓一体

（作者：尚硅谷研究院）

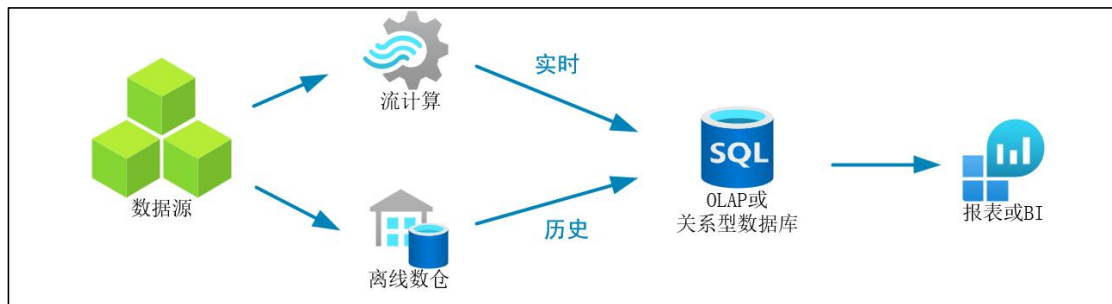
版本：V1.0

第 1 章 什么要做湖仓一体

从目前业界整体的湖仓一体的实践来看，主要的目的就是解决之前 lambda 和 kappa 架构的问题。通过对利用数据湖的特点，重组了流处理与批处理，使得二者不再冗余与冲突，形成真正的流批一体，从而实现降本增效。

那首先来看看什么是 lambda 和 kappa 架构

1) lambda 架构:

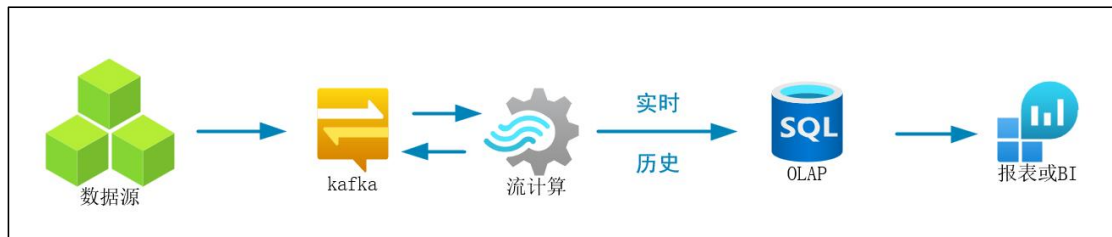


lambda 架构算是流批一体架构的 1.0 版本，实际上就是利用流计算生产当日的数据，利用离线批处理计算当日以前的数据，最后在 OLAP 数据库中进行合并汇总，产生报告。

对于数据需求方来说，感知不到数据是通过两条途径获得的。

至今依然有很多企业使用这种架构。但问题也是很明显的就是，开发成本较高，要同时维护两条计算链路，而且还要保证两者的口径一致计算逻辑统一，占用并维护两套计算资源，都是存在巨大的成本和效率问题的。

2) kappa 架构:



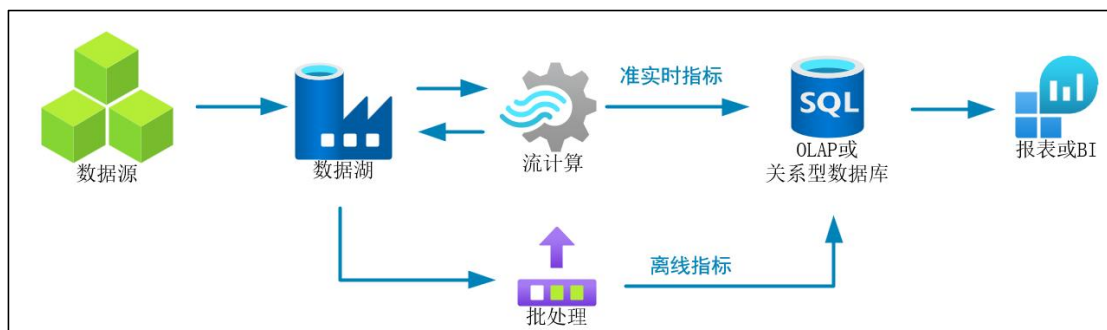
kappa 架构算是流批一体的 2.0 版本，它的核心思路就是为了避免出现两套计算链路，而完全依赖流式计算。把 kafka 的地位从一个数据队列的地位，提高到了一个流式的数据容器，配合流式计算引擎形成了一个实时的数据仓库，同时 OLAP 也承担了更多历史数据的数据的工作。最终只保留一套计算链路，避免了计算资源和开发资源的冗余。

这套架构的问题在于：当面临海量数据和大量指标时，所有指标都靠 Flink 进行计算，海量数据都使用 kafka 和 OLAP 进行存储，这个硬件成本开销是远大于离线数仓的。

而事实上绝大多数的指标其实并不需要非常实时（秒级）的结果。

而且 kafka 虽然存储了大量数据，但其只能作为流计算的中间节点，没有任何分析能力。

3) 湖仓一体架构：



湖仓一体算是流批一体的 3.0 版本，数据湖既可以像 kafka 那样作为中间存储容器推动流计算，又因为数据湖底层是保存在 hdfs 中，所以它还可以兼容离线的处理方案。

这样就可以根据指标的时效特点，来区分对待：准实时指标使用 Flink 计算，离线指标还维持原来的低成本的批处理，用一套存储兼容两种方案。

这种框架的问题：

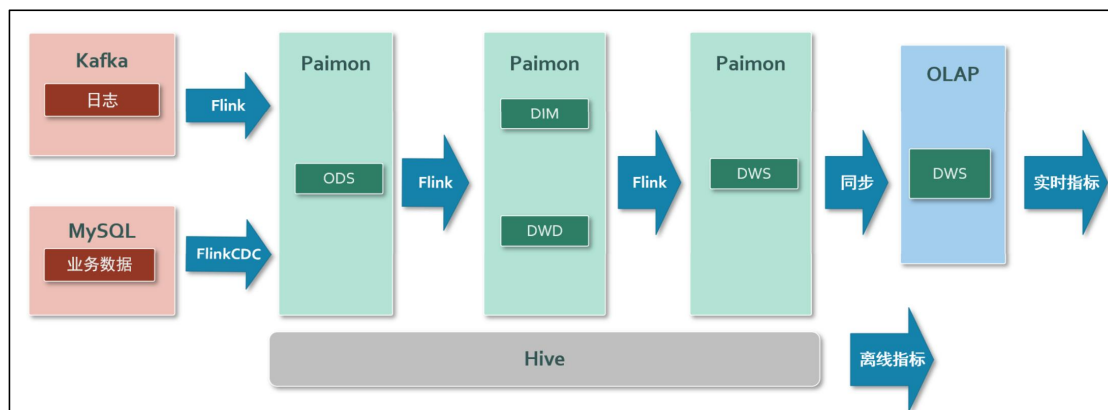
首先数据湖和 kafka 还存在性能上的差距，一般只做十秒级或分钟级指标的计算。

目前的数据湖开源框架和传统离线与实时框架成熟度还有差距，开源社区的活跃度当然也有一定差距。

网上的资料和项目案例非常少。

第 2 章 湖仓一体整体架构

数据处理流程如下：



该流程的一些特点：

- 整体流程非常依赖 Flink，从数据采集到数据分层加工，一直到写入 doris 都是由 Flink 完成。
- 该过程完全流式计算，数据延迟，考虑到数据计算压力，一般推荐时效到分钟级。
- 全流程大部分计算都是用 FlinkSQL 完成。
- DWS 层同步写入到 OLAP 数据库是为了能够更快速的实现即席查询。
- 全流程在 Paimon 中的全部表，都可以被 Hive 查询、计算。考虑到兼容已开发的离线计算脚本或者降低流式计算成本。可以允许一定的 Hive 离线计算。

第 3 章 Paimon 简介

3.1 简介

Flink 社区希望能够将 Flink 的 Streaming 实时计算能力和 Lakehouse 新架构优势进一步结合，推出新一代的 Streaming Lakehouse 技术，促进数据在数据湖上真正实时流动起来，并为用户提供实时离线一体化的开发体验。Flink 社区内部孵化了 Flink Table Store（简称 FTS）子项目，一个真正面向 Streaming 以及 Realtime 的数据湖存储项目。2023 年 3 月 12 日，FTS 进入 Apache 软件基金会（ASF）的孵化器，改名为 Apache Paimon (incubating)。

Apache Paimon 是一个流数据湖平台，具有高速数据摄取、变更日志跟踪和高效的实时分析的能力。

3.2 核心特性

1) 统一批处理和流处理

批量写入和读取、流式更新、变更日志生成，全部支持。

2) 低存储成本

所有存储都保存在 hdfs，存储扩展性强。

3) 支持各种变更

支持数据的 upsert, update, delete 的变更操作，同时也支持数据结构的 alter 变更操作。

4) 变更日志生成

Apache Paimon 可以从任何数据源生成正确且完整的变更日志，从而简化您的流分析。

5) 丰富的表类型

支持可以去重的主键表，only-append 的流水表，和聚合表。

6) 外部生态的支持

目前已有不少 olap 支持 paimon 作为外表进行查询，对于数据联合查询和同步都非常方便。

第 4 章 湖仓一体环境搭建

目前已安装的软件环境

软件	版本
hadoop	3.3.4
hive	3.1.3
spark	3.3.2
doris	2.1.0
flink	1.17.2
kafka	3.3.1
zookeeper	3.7.1
mysql	8.0.31

4.1 启动 Hadoop 集群

(1) 启动 hadoop 集群。

(2) 启动 hive metastore，用于保存 paimon 的元数据

```
nohup hive --service metastore &
```

4.2 安装 Paimon 集群

拷贝 paimon 的两个 jar 包到 flink 的 lib 目录下

```
cp paimon-flink-1.17-0.8-20240503.002102-77.jar
/opt/module/flink-1.17.2/lib
cp paimon-flink-action-0.8-20240503.002102-77.jar
/opt/module/flink-1.17.2/lib
```

4.3 创建流处理 flink 客户端脚本并初始化目录

flink-sql 客户端初始化文件

1) 创建初始化 sql 文件。

```
vim /opt/module/flink-1.17.2/conf/sql-client-init.sql
```

2) 元数据目录保存在 hdfs（不推荐）。

```
CREATE CATALOG fs_catalog WITH (
  'type' = 'paimon',
  'warehouse' = 'hdfs://hadoop102:8020/paimon/fs'
);
```

3) 元数据保存在 hive（推荐）。

```
CREATE CATALOG hive_catalog WITH (
  'type' = 'paimon',
  'metastore' = 'hive',
  'uri' = 'thrift://hadoop102:9083',
  'hive-conf-dir' = '/opt/module/hive/conf',
  'warehouse' = 'hdfs://hadoop102:8020/paimon/hive'
);
USE CATALOG hive_catalog;
SET 'sql-client.execution.result-mode' = 'tableau';
```

4) 启动 sql-client 时，指定该 sql 初始化文件

```
bin/sql-client.sh -s yarn-session -i conf/sql-client-init.sql
```

5) 查看 catalog

```
show catalogs;
show current catalog;
```

4.4 创建批处理 flink 客户端脚本并初始化目录

1) 创建初始化 sql 文件。

```
vim /opt/module/flink-1.17.2/conf/sql-client-init-batch.sql
```

2) 元数据目录保存在 hdfs（不推荐）。

```
CREATE CATALOG fs_catalog WITH (
  'type' = 'paimon',
  'warehouse' = 'hdfs://hadoop102:8020/paimon/fs'
);
```

3) 元数据保存在 hive（推荐）。

```
CREATE CATALOG hive_catalog WITH (
  'type' = 'paimon',
  'metastore' = 'hive',
  'uri' = 'thrift://hadoop102:9083',
  'hive-conf-dir' = '/opt/module/hive/conf',
  'warehouse' = 'hdfs://hadoop102:8020/paimon/hive'
);
USE CATALOG hive_catalog;
```

```
SET 'sql-client.execution.result-mode' = 'tableau';
RESET 'execution.checkpointing.interval';
SET 'execution.runtime-mode' = 'batch';
```

4) 启动 sql-client 时，指定该 sql 初始化文件

创建一个在/home/atguigu/bin 下创建 flink-sql-batch

```
bin/sql-client.sh -s yarn-session -i conf/sql-client-init.sql
```

5) 查看 catalog

```
show catalogs;
show current catalog;
```

第 5 章 业务数据入湖

5.1 需要准备的 jar 包

把 `flink-sql-connector-mysql-cdc-2.4.2.jar` 和

`paimon-hive-connector-3.1-0.8-20240503.002102-76.jar` 拷入 `flink-1.17.2/lib/` 目录下

5.2 flink 环境配置

flink-conf.yaml

每个任务启动时的默认值

```
jobmanager.memory.process.size: 768m
taskmanager.memory.process.size: 1024m
taskmanager.numberOfTaskSlots: 2
parallelism.default: 2
execution.checkpointing.interval: 15s
state.backend.type: rocksdb
state.checkpoints.dir: hdfs://hadoop102:8020/flink-checkpoints
state.backend.incremental: true
```

5.3 创建为数据湖创建数据库

(1) 启动 yarn-session，用于执行客户端命令行内的各种查询任务。

```
[atguigu@hadoop102 bin]$ yarn-session.sh -d
```

(2) 编写一个简易的启动 flink 客户端的脚本

```
[atguigu@hadoop102 bin]$ vim flink-sql
```

(3) 编辑脚本（复制到 flink-sql 中）

```
/opt/module/flink-1.17.2/bin/sql-client.sh -s yarn-session -i
/opt/module/flink-1.17.2/conf/sql-client-init.sql
```

(4) 赋权限

```
[atguigu@hadoop102 bin]$ sudo chmod +x flink-sql
```

(5) 启动客户端

```
[atguigu@hadoop102 bin]$ flink-sql
```

(6) 创建数据库

```
Flink SQL> create database gmall_lake2024
```

5.4 同步表

```
flink run-application -t yarn-application \  
  --detached \  
  -Dyarn.application.name=gmall_lake2024_cdc \  
  -Dparallelism.default=2 \  
  -Djobmanager.memory.process.size=1024mb \  
  -Dtaskmanager.memory.process.size=2048mb \  
  -Dtaskmanager.numberOfTaskSlots=2 \  
  
/opt/module/flink-1.17.2/lib/paimon-flink-action-0.8-20240503.002102-77.jar \  
  mysql-sync-database \  
  --warehouse hdfs://hadoop102:8020/paimon/hive \  
  --database gmall_lake2024 \  
  --table-prefix "ods_" \  
  --table-suffix "_cdc" \  
  --mysql_conf hostname=hadoop102 \  
  --mysql_conf username=root \  
  --mysql_conf password=000000 \  
  --mysql_conf database-name=gmall \  
  --catalog-conf metastore=hive \  
  --catalog-conf uri=thrift://hadoop102:9083 \  
  --table-conf bucket=2 \  
  --table-conf changelog-producer=input \  
  --table-conf sink.parallelism=2 \  
  --including-tables  
'user_info|sku_info|base_trademark|base_province'
```

其中蓝色部分是 flink 自带的参数，

- 红色部分是执行的 flink 程序包(该包由 paimon 官方制作)，
- 绿色是执行包中的子命令，
- 紫色是该子命令需要的参数

paimon-flink-action-xxx.jar 是 paimon 官方提供的 flink 任务工具包，里面封装了多种 Flink 程序任务包括：各种 Cdc 工具，手动 Compaction，数据迁移，手动删除分区，手动创建 Tag，删除 Tag 等等操作。

子命令参数说明：

配置	描述
--warehouse	Paimon 仓库路径。
--database	Paimon Catalog 中的数据库名称。
--mysql-conf	Flink CDC MySQL 源表的配置。每个配置都应以“key=value”的格式指定。主机名、用户名、密码、数据库名和表名是必需

	配置，其他是可选配置。
--catalog-conf	Paimon Catalog 的配置。 <code>metastore</code> 为元数据类型， <code>uri</code> 为元数据访问地址
--table-conf	Paimon 表 sink 的配置。 <code>bucket</code> 是分桶数， <code>changelog-producer=input</code> 适用于数据源输入本身就有变更日志， <code>sink.parallelism</code> 是写入并行度
--table-conf changelog-producer	这个参数用于为该表产生流式变更日志，用于下游流式计算。 <code>none</code>: 只保留 snapshot 级的变更日志，会丢失一部分中间变化。虽然 <code>flink</code> 本身也会辅助生成 <code>changelog</code> 但是开销太大不推荐。 <code>input</code>: 保留上游输入的变更日志，一般上游输入为 CDC 时使用。 <code>lookup</code>: 在上游没有输入 CDC 日志时，依据 <code>paimon</code> 对数据的对比生成新的 <code>changelog</code> 供下游使用，在上游非 <code>cdc</code> 任务时，算是最准确的 <code>changelog</code>。 <code>full-compaction</code>: 相比 <code>lookup</code> 生成 <code>changelog</code> 间隔较大，可以通过设置 <code>compaction</code> 间隔，在 <code>compaction</code> 时产生最新的 <code>changelog</code> 给下游，性能相对 <code>lookup</code> 较好，但产生延迟受 <code>compaction</code> 影响。
--including-tables	对应 <code>mysql</code> 中的表名，用 <code> </code> 分隔

注意：涉及 `cdc` 同步的表，必须有主键。

如果指定的 Paimon 表不存在，此操作将自动创建该表。其 `schema` 将从所有指定的 MySQL 表派生。如果 Paimon 表已存在，则其 `schema` 将与所有指定 MySQL 表的 `schema` 进行比较。

5.5 查验数据

5.5.1 HDFS 上查看

去 HDFS 上查看是否有数据到达。

/paimon/hive/gmall_lake2024.db

Go!

Show

25

entries

Search:

☐		Permission		Owner		Group		Size		Last Modified		Replication		Block Size		Name	
☐		drwxr-xr-x		atguigu		supergroup		0 B		May 07 17:36		0		0 B		ods_base_category3_cdc	
☐		drwxr-xr-x		atguigu		supergroup		0 B		May 07 17:36		0		0 B		ods_base_dic_cdc	
☐		drwxr-xr-x		atguigu		supergroup		0 B		May 07 17:36		0		0 B		ods_base_province_cdc	
☐		drwxr-xr-x		atguigu		supergroup		0 B		May 07 17:36		0		0 B		ods_base_trademark_cdc	
☐		drwxr-xr-x		atguigu		supergroup		0 B		May 07 17:36		0		0 B		ods_sku_info_cdc	
☐		drwxr-xr-x		atguigu		supergroup		0 B		May 07 17:36		0		0 B		ods_user_info_cdc	

Showing 1 to 6 of 6 entries

Previous

1

Next

5.5.2 在 flink 客户端查看

可以在 flink-sql-batch 中查看表。

```
Flink SQL> use gmall_lake2024;
[INFO] Execute statement succeed.
```

```
Flink SQL> show tables;
+-----+
|          table name          |
+-----+
| ods_base_category3_cdc |
|   ods_base_dic_cdc   |
| ods_base_province_cdc |
| ods_base_trademark_cdc |
|   ods_sku_info_cdc   |
|   ods_user_info_cdc   |
+-----+
6 rows in set
```

查看表中数据。

```
Flink SQL> select * from ods_base_province_cdc ;
```

5.6 为任务增加新的表

5.6.1 停止任务，保存断点

停止任务的时候进行存盘。

```
[atguigu@hadoop102 bin]$ flink stop -p
hdfs://hadoop102:8020/paimon/savepoint/gmall_lake2024_cdc -yid
application_1713772881362_0004 2e9796ad5ae5d2009d04072dc095bf0a
```

这里要填写任务存盘的地址、yarn 的 appid 和 flink 的 jobid。

```
2024-04-24 17:50:51,345 INFO org.apache.flink.yarn.YarnClusterDescriptor [] - Found Web Interface
on 'application_1713772881362_0004'.
Savepoint completed. Path: hdfs://hadoop102:8020/paimon/savepoint/gmall_lake2024_cdc/savepoint-2e9796-7ca4e80a0d1f
[atguigu@hadoop102 bin]$
```

执行完成后会有如上提示。

5.6.2 开始新任务，加载存盘

```
flink run-application -t yarn-application \
--detached \
```

```

--fromSavepoint
hdfs://hadoop102:8020/paimon/savepoint/gmall_lake2024_cdc/savepoi
nt-2e9796-7ca4e80a0d1f \
-Dyarn.application.name=gmall_lake2024_cdc \
-Dparallelism.default=2 \
-Djobmanager.memory.process.size=1024mb \
-Dtaskmanager.memory.process.size=2048mb \
-Dtaskmanager.numberOfTaskSlots=2 \
--allowNonRestoredState \

/opt/module/flink-1.17.2/lib/paimon-flink-action-0.8-20240503.002
102-77.jar \
mysql-sync-database \
--warehouse hdfs://hadoop102:8020/paimon/hive \
--database gmall_lake2024 \
--table-prefix "ods_" \
--table-suffix "_cdc" \
--mysql-conf hostname=hadoop102 \
--mysql-conf username=root \
--mysql-conf password=000000 \
--mysql-conf database-name=gmall \
--catalog-conf metastore=hive \
--catalog-conf uri=thrift://hadoop102:9083 \
--table-conf bucket=2 \
--table-conf changelog-producer=input \
--table-conf sink.parallelism=2 \
--including-tables
'user_info|sku_info|base_trademark|base_province|base_category3|b
ase_dic'
    
```

这里增加了要同步的表， 还要加载的进度。

--allowNonRestoredState 是允许之前存在之前没有状态的算子节点。

Overview

Exceptions

TimeLine

Checkpoints

Configuration

Overview

History

Summary

Configuration

Checkpoint Counts

Triggered: 1

In Progress: 0

Completed: 0

Failed: 1

Restored: 1

Latest Completed Checkpoint

None

Latest Failed Checkpoint

None

Latest Savepoint

None

Latest Restore

ID: 580

Restore Time: 2024-04-24 18:50:01

Type: Savepoint

Path: hdfs://hadoop102:8020/paimon/savepoint/gmall_lake2024_cdc/savepoint-2e9796-7ca4e80a0d1f

5.7 自定义字段分区表采集

5.7.1 已拥有直接可用的分区字段的表

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

作为有较大数据量的事实表，通常会采取分区的方式进行写入。

如果有字段可以直接作为分区字段，比如打算按日分区，而字段中有 `dt` 或者 `create_date` 这种不需要加工直接可以用作分区字段的字段。那依然可以沿用上一节的方式利用官方提供的工具包，使用 `mysql-sync-database` 命令进行多表采集。只不过要提前在 `ods` 层建好分区表

```
CREATE TABLE MyTable (
  user_id BIGINT,
  item_id BIGINT,
  behavior STRING,
  dt STRING,
  hh STRING,
  PRIMARY KEY (dt,user_id) NOT ENFORCED
) PARTITIONED BY (dt);
```

注意：主键必须包含分区。

5.7.2 未有直接可用的分区字段的表

如果来源表中并没有包含现成的分区字段，比如时间字段是 `ts` 或者 `create_time`，需要简单加工一下，那就需要使用工具包的 `mysql-sync-table` 命令，以表为单位进行采集。

1) 订单表:

```
flink run-application -t yarn-application \
  --detached \
  -Dyarn.application.name=gmall_lake2024_cdc \
  -Dparallelism.default=2 \
  -Djobmanager.memory.process.size=1024mb \
  -Dtaskmanager.memory.process.size=2048mb \
  -Dtaskmanager.numberOfTaskSlots=2 \
  --allowNonRestoredState \

/opt/module/flink-1.17.2/lib/paimon-flink-action-0.8-20240503.002
102-77.jar \
  mysql_sync_table \
  --warehouse hdfs://hadoop102:8020/paimon/hive \
  --database gmall_lake2024 \
  --table ods_order_info_cdc \
  --partition_keys dt \
  --primary_keys dt,id \
  --computed_column 'dt=date_format(create_time, 'yyyy-MM-dd')' \
  \
  --mysql_conf hostname=hadoop102 \
  --mysql_conf username=root \
  --mysql_conf password=000000 \
  --mysql_conf database-name=gmall \
  --mysql_conf table-name='order_info' \
  --catalog_conf metastore=hive \
  --catalog_conf uri=thrift://hadoop102:9083 \
  --table_conf bucket=2 \
  --table_conf changelog-producer=input \
  --table_conf sink.parallelism=2 \
  --table_conf metastore.partitioned-table=true
```

这个以表为单位的导入作业，除了要填写来源和目标表名以外，还声明了主键和分区字段。

但是其中 dt 这个字段在原表中并不存在，是通过 date_format 函数计算得来的，并且利用--computed_column 作为一个计算产生的字段加入到新表中。于是添加参数：

```
--computed_column 'dt=date_format(create_time,'yyyy-MM-dd')
```

--table_conf metastore.partitioned-table=true 这个参数主要是用于 hive 兼容 paimon 的分区，如果不考虑 hive 该参数可以不设置。

2) 订单明细表:

```
flink run-application -t yarn-application \  
  --detached \  
  -Dyarn.application.name=gmall_lake2024_cdc \  
  -Dparallelism.default=2 \  
  -Djobmanager.memory.process.size=1024mb \  
  -Dtaskmanager.memory.process.size=3072mb \  
  -Dtaskmanager.numberOfTaskSlots=2 \  
  --allowNonRestoredState \  
  
/opt/module/flink-1.17.2/lib/paimon-flink-action-0.8-20240503.002102-77.jar \  
  mysql_sync_table \  
  --warehouse hdfs://hadoop102:8020/paimon/hive \  
  --database gmall_lake2024 \  
  --table ods_order_detail_cdc \  
  --partition_keys dt \  
  --primary_keys dt,id \  
  --computed_column 'dt=date_format(create_time,'yyyy-MM-dd')' \  
 \  
  --mysql_conf hostname=hadoop102 \  
  --mysql_conf username=root \  
  --mysql_conf password=000000 \  
  --mysql_conf database-name=gmall \  
  --mysql_conf table-name='order_detail' \  
  --catalog_conf metastore=hive \  
  --catalog_conf uri=thrift://hadoop102:9083 \  
  --table_conf bucket=2 \  
  --table_conf changelog-producer=input \  
  --table_conf sink.parallelism=2 \  
  --table_conf metastore.partitioned-table=true
```

5.8 在 flink-sql 客户端中查看表

```
Flink SQL> show tables;
+-----+
|               table name               |
+-----+
| ods_base_category3_cdc |
| ods_base_dic_cdc      |
| ods_base_province_cdc |
| ods_base_trademark_cdc|
| ods_order_detail_cdc  |
| ods_order_info_cdc    |
| ods_sku_info_cdc      |
| ods_user_info_cdc     |
+-----+
8 rows in set

Flink SQL>
```

查看生成的表结构:

```
Flink SQL> show create table ods_order_info_cdc ;
```

```
CREATE TABLE `hive_catalog`.`gmall_lake2024`.`ods_order_info_cdc` (
  `id` BIGINT NOT NULL,
  `consignee` VARCHAR(100),
  `consignee_tel` VARCHAR(20),
  `total_amount` DECIMAL(10, 2),
  `order_status` VARCHAR(20),
  `user_id` BIGINT,
  `payment_way` VARCHAR(20),
  `delivery_address` VARCHAR(1000),
  `order_comment` VARCHAR(200),
  `out_trade_no` VARCHAR(50),
  `trade_body` VARCHAR(200),
  `create_time` TIMESTAMP(0),
  `operate_time` TIMESTAMP(0),
  `expire_time` TIMESTAMP(0),
  `process_status` VARCHAR(20),
  `tracking_no` VARCHAR(100),
  `parent_order_id` BIGINT,
  `img_url` VARCHAR(200),
  `province_id` INT,
  `activity_reduce_amount` DECIMAL(16, 2),
  `coupon_reduce_amount` DECIMAL(16, 2),
  `original_total_amount` DECIMAL(16, 2),
  `feight_fee` DECIMAL(16, 2),
  `feight_fee_reduce` DECIMAL(16, 2),
  `refundable_time` TIMESTAMP(0),
  `dt` VARCHAR(2147483647) NOT NULL,
  CONSTRAINT `PK_dt_id` PRIMARY KEY (`dt`, `id`) NOT ENFORCED
) COMMENT '订单表 订单表'
PARTITIONED BY (`dt`)
WITH (
  'bucket' = '2',
  'path' = 'hdfs://hadoop102:8020/paimon/hive/gmall_lake2024.db/ods_order_info_cdc',
  'changelog-producer' = 'input',
  'sink.parallelism' = '2',
  'metastore.partitioned-table' = 'true'
)
```

```
Flink SQL> show create table ods_order_detail_cdc ;
```

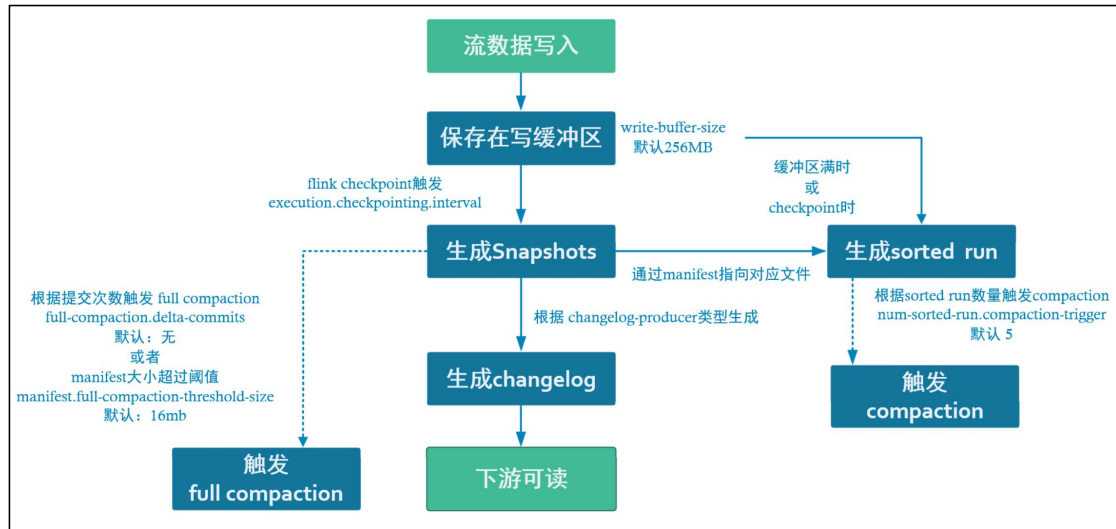
```
| CREATE TABLE `hive_catalog`.`gmall_lake2024`.`ods_order_info_cdc` (  
  `id` BIGINT NOT NULL,  
  `consignee` VARCHAR(100),  
  `consignee_tel` VARCHAR(20),  
  `total_amount` DECIMAL(10, 2),  
  `order_status` VARCHAR(20),  
  `user_id` BIGINT,  
  `payment_way` VARCHAR(20),  
  `delivery_address` VARCHAR(1000),  
  `order_comment` VARCHAR(200),  
  `out_trade_no` VARCHAR(50),  
  `trade_body` VARCHAR(200),  
  `create_time` TIMESTAMP(0),  
  `operate_time` TIMESTAMP(0),  
  `expire_time` TIMESTAMP(0),  
  `process_status` VARCHAR(20),  
  `tracking_no` VARCHAR(100),  
  `parent_order_id` BIGINT,  
  `img_url` VARCHAR(200),  
  `province_id` INT,  
  `activity_reduce_amount` DECIMAL(16, 2),  
  `coupon_reduce_amount` DECIMAL(16, 2),  
  `original_total_amount` DECIMAL(16, 2),  
  `feight_fee` DECIMAL(16, 2),  
  `feight_fee_reduce` DECIMAL(16, 2),  
  `refundable_time` TIMESTAMP(0),  
  `dt` VARCHAR(2147483647) NOT NULL,  
  CONSTRAINT `PK_dt_id` PRIMARY KEY (`dt`, `id`) NOT ENFORCED  
) COMMENT '订单表 订单表'  
PARTITIONED BY (`dt`)  
WITH (  
  'bucket' = '2',  
  'path' = 'hdfs://hadoop102:8020/paimon/hive/gmall_lake2024.db/ods_order_info_cdc',  
  'changelog-producer' = 'input',  
  'sink.parallelism' = '2',  
  'metastore.partitioned-table' = 'true'  
)
```

检查数据

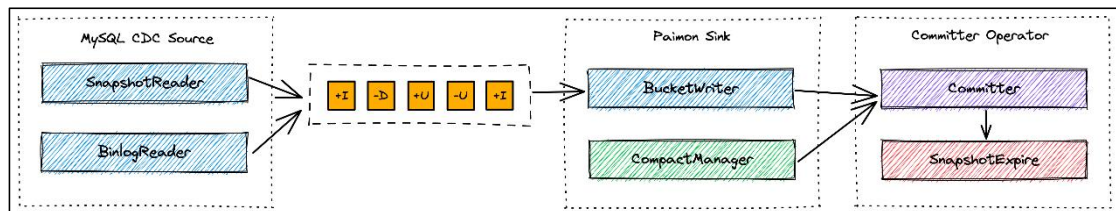
```
Flink SQL> select * from ods_order_info_cdc limit 10;
```

第 6 章 理解 Flink 写入数据流程

6.1 流程图

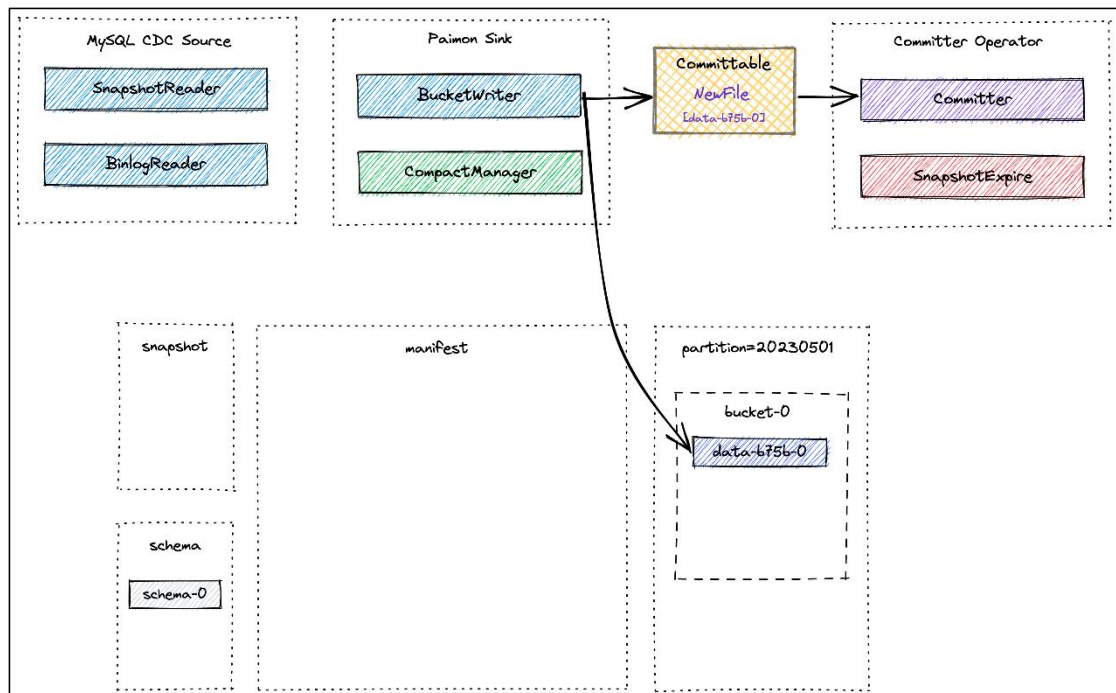


6.2 数据写入



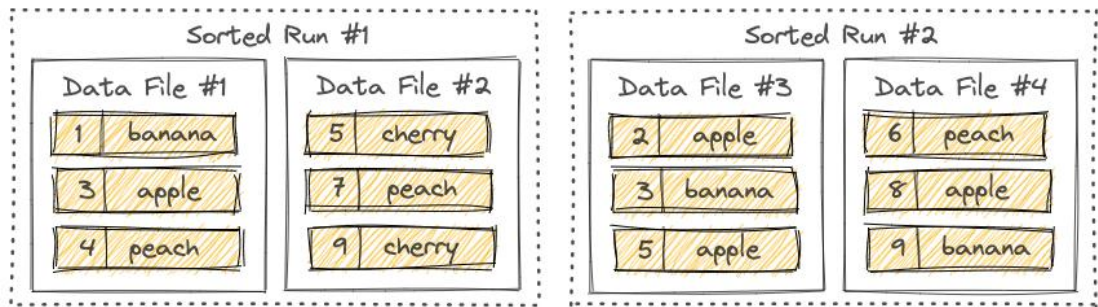
MySQL Cdc Source 读取快照和增量数据，并在规范化后将它们发送到下游。

6.3 生成文件



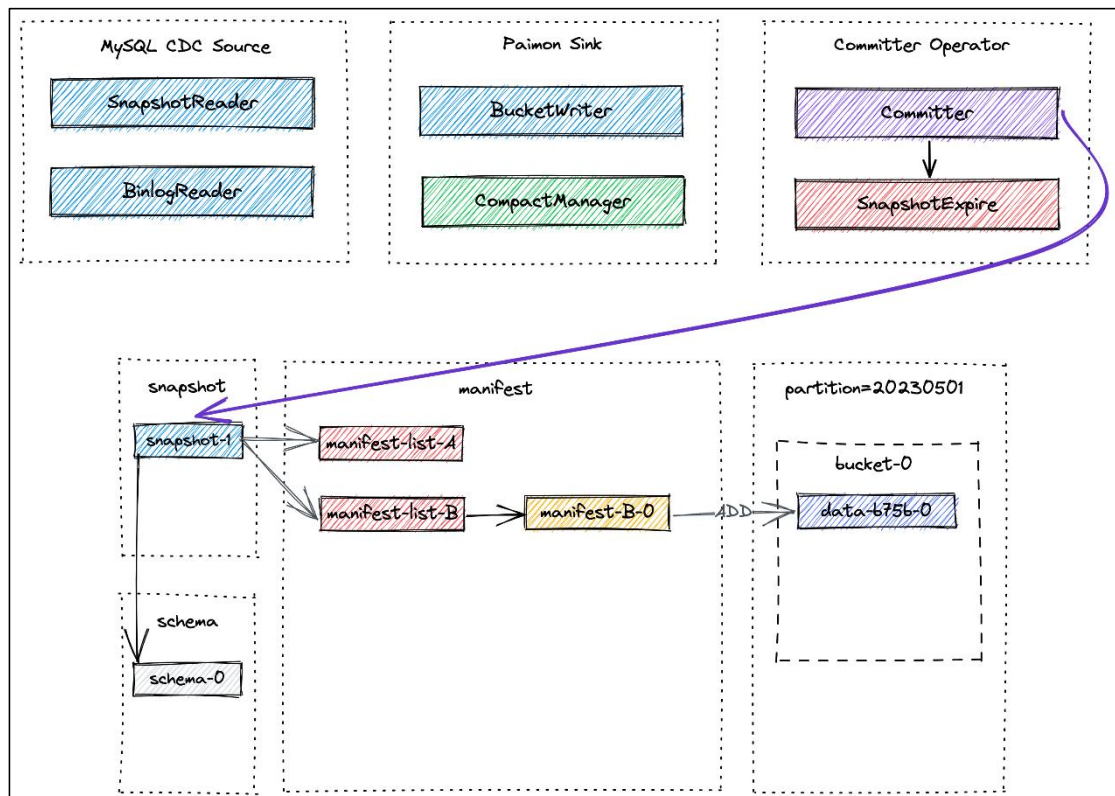
Paimon Sink 首先将新记录缓冲在基于堆的 LSM 树中，并在内存缓冲区满时将它们刷新到磁盘。请注意，写入的每个数据文件都是 sorted run。此时，还没有 manifest file 和 snapshot。在 Flink 检查点发生之前，Paimon Sink 将刷新所有缓冲记录并向下游发送可提交消息，该消息在检查点期间由 Committer Operator 读取并提交。

什么是 sorter run



sorted run(有序段)，每次 checkpoint 会产生一个或多个 sorted run(每满一次缓冲区会产生一个 sorted run),每个 sorted run 包含一个或多个文件(主要和数据分区分桶有关系)，每个数据文件内部主键唯一且有序,默认使用 orc 格式。

6.4 创建 snapshot



在保存 checkpoint 的时候，Committer Operator 将创建一个新快照并将其与 manifest list 关联起来，使得快照通过 manifest 关联到了数据文件。

manifest 的目的就是为快照指向哪些数据文件。

manifest 包括两种文件 manifest lists 和 manifest files，保存在 manifest 目录中。

每个快照的 manifest list 由两个文件组成一个是 base 文件，一个是 delta 文件。

每个快照文件会产生一个 snapshot 文件（如下的 json 文件），其中会记录两个新产生的 manifest lists 文件一个是 base 文件，一个是 delta 文件。（如果是首个快照只会有 delta）

snapshot 的 json 文本文件

```
{
  "version" : 3,
  "id" : 1,
  "schemaId" : 0,
  "baseManifestList" : "manifest-list-4ccc-c07f-4090-958c-cfe3ce3889e5-0",
  "deltaManifestList" : "manifest-list-4ccc-c07f-4090-958c-cfe3ce3889e5-1",
  "changelogManifestList" : null,
  "commitUser" : "7d758485-981d-4b1a-a0c6-d34c3eb254bf",
  "commitIdentifier" : 9223372036854775807,
  "commitKind" : "APPEND",
  "timeMillis" : 1684155393354,
  "logOffsets" : { },
  "totalRecordCount" : 1,
  "deltaRecordCount" : 1,
  "changelogRecordCount" : 0,
  "watermark" : -9223372036854775808
}
```

比如上方的 baseManifestList 和 deltaManifestList 分别指向了两个文件。

base 指向的 manifest list 文件会记录本次快照是以哪些 manifest file 以为基础的。

delta 指向的 manifest list 文件会记录本次快照是新增了哪些 manifest file。

与 manifest list 不同 manifest file，并不都是本次快照产生的，可能是以前的产生的，但是被本次快照继承了下来，一般 base 关联的都是旧的，delta 关联的都是新产生的。

每个 manifest file 都关联了一个或多个具体的数据文件。

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
-rw-r--r--	atguigu	supergroup	5.24 KB	May 20 15:49	3	128 MB	manifest-24703349-fc41-4440-892b-9fff54f0a8ef-0
-rw-r--r--	atguigu	supergroup	5.25 KB	May 20 15:50	3	128 MB	manifest-24703349-fc41-4440-892b-9fff54f0a8ef-1
-rw-r--r--	atguigu	supergroup	1.85 KB	May 20 16:49	3	128 MB	manifest-24703349-fc41-4440-892b-9fff54f0a8ef-2
-rw-r--r--	atguigu	supergroup	1.87 KB	May 20 16:49	3	128 MB	manifest-24703349-fc41-4440-892b-9fff54f0a8ef-3
-rw-r--r--	atguigu	supergroup	1.86 KB	May 20 16:59	3	128 MB	manifest-24703349-fc41-4440-892b-9fff54f0a8ef-4
-rw-r--r--	atguigu	supergroup	1.86 KB	May 20 16:59	3	128 MB	manifest-24703349-fc41-4440-892b-9fff54f0a8ef-5
-rw-r--r--	atguigu	supergroup	4.37 KB	May 08 11:08	3	128 MB	manifest-91ced66d-1462-476d-a3b4-b36aad50f76c-0
-rw-r--r--	atguigu	supergroup	4.39 KB	May 08 11:08	3	128 MB	manifest-91ced66d-1462-476d-a3b4-b36aad50f76c-1
-rw-r--r--	atguigu	supergroup	676 B	May 08 11:08	3	128 MB	manifest-list-92ff929-5f64-46d0-93c9-cda08881a597-0
-rw-r--r--	atguigu	supergroup	813 B	May 08 11:08	3	128 MB	manifest-list-92ff929-5f64-46d0-93c9-cda08881a597-1
-rw-r--r--	atguigu	supergroup	813 B	May 08 11:08	3	128 MB	manifest-list-92ff929-5f64-46d0-93c9-cda08881a597-2
-rw-r--r--	atguigu	supergroup	813 B	May 08 11:08	3	128 MB	manifest-list-92ff929-5f64-46d0-93c9-cda08881a597-3
-rw-r--r--	atguigu	supergroup	676 B	May 08 11:08	3	128 MB	manifest-list-92ff929-5f64-46d0-93c9-cda08881a597-4
-rw-r--r--	atguigu	supergroup	813 B	May 08 11:08	3	128 MB	manifest-list-92ff929-5f64-46d0-93c9-cda08881a597-5

在客户端中查看某张表已有的快照

```
SELECT * FROM ods_base_province_cdc$snapshots;
```

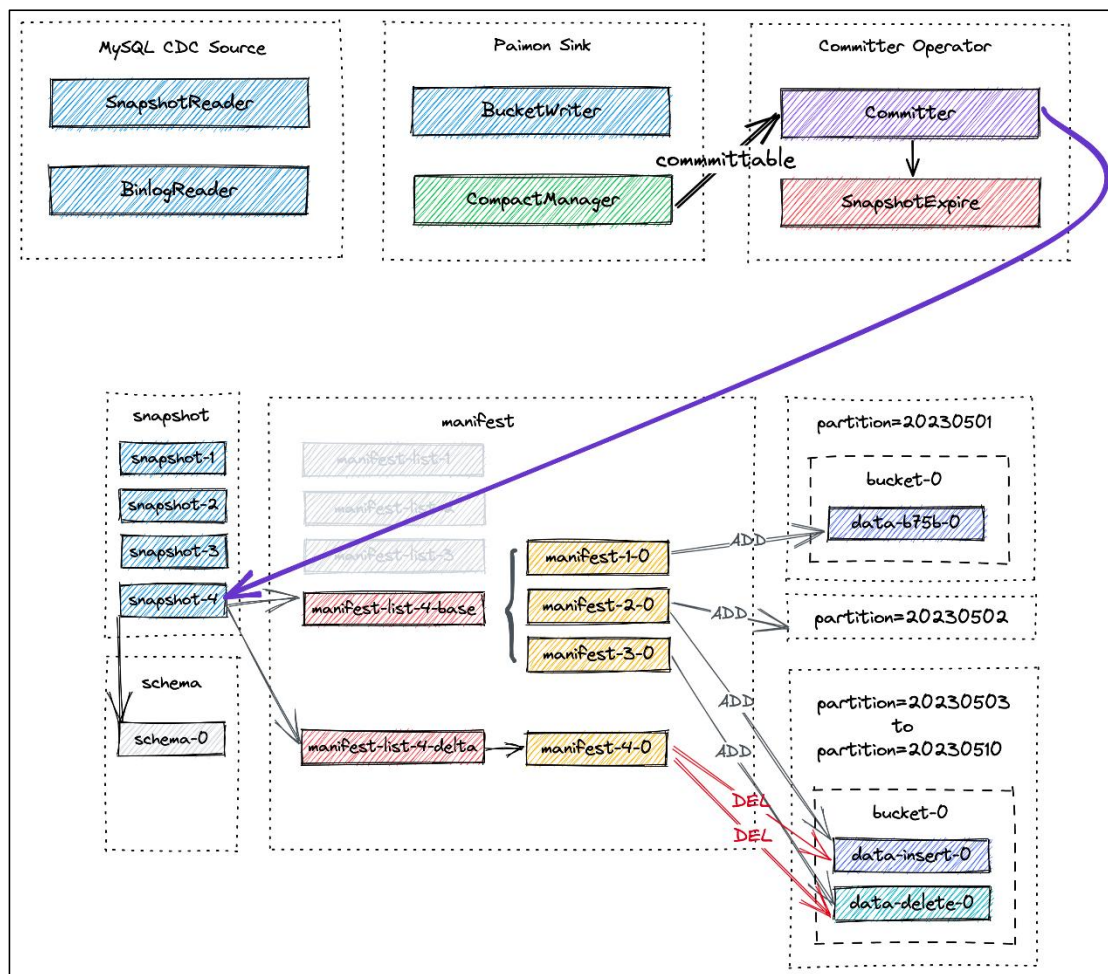
snapshot_id	schema_id	commit_user	commit_identifier	commit_kind	commit_time	base_manifest_list	delta_manifest_list	changelog_manifest_list	total_record_count	delta_record_count	changelog_record_count	watermark
1	0	b4e3eecd-48ec-47b9-8ad6-0bd...	1	APPEND	2024-05-20 15:38:00.768	manifest-list-8f363f22-3497...	manifest-list-8f363f22-3497...	manifest-list-8f363f22-3497...	34	34	34	-9223372036854775808
2	0	b4e3eecd-48ec-47b9-8ad6-0bd...	2	APPEND	2024-05-20 15:38:13.666	manifest-list-8f363f22-3497...	manifest-list-8f363f22-3497...	manifest-list-8f363f22-3497...	34	0	0	-9223372036854775808
3	0	b4e3eecd-48ec-47b9-8ad6-0bd...	1703	APPEND	2024-05-21 00:10:14.967	manifest-list-8f363f22-3497...	manifest-list-8f363f22-3497...	manifest-list-8f363f22-3497...	34	0	0	-9223372036854775808

可以利用 snapshot_id 进行“时空穿越”式的查询

```
SELECT * FROM ods_base_province_cdc /*+ OPTIONS('scan.snapshot-id' = '1') */;
```

6.5 Compaction 异步压缩

由于多个 sorted run 会产生小文件，所以会对新生成的数据进行压缩合并。



Paimon 使用 LSM 树，支持大量更新。LSM 在多个 sorted run 中组织文件。从 LSM 树查询记录时，必须组合所有排序运行以生成所有记录的完整视图。

当 `num-sorted-run.compaction-trigger` 变大时，Compaction 会得不那么频繁，从而提高写入性能。但是，如果该值变得太大，则查询表时将需要更多内存和 CPU 时间。这是写入和查询性能之间的权衡。

6.6 Full Compaction

Full Compaction 是比 Compaction 更重的合并，是针对整个表的全部数据进行整理，彻底讲所有小文件合并，提高整体存储效率。

但是相对的 Full compaction 的付出的执行代价也较大，所以执行的频率也要相对控制。

目前已有主要控制参数为 `manifest.full-compaction-threshold-size` 这个是 manifest 的总大小。更有效的直接控制是通过 `full-compaction.delta-commits` 目前没有默认值，生产上可以设置几十到上百。

6.7 生成 changelog

下游流式读取某长辈的数据主要不是看 snapshot，而是看 changelog，不同的 changelog 生成方式决定了下游能够读取的数据内容和时机。

目前的 changelog 的生成方式主要看 changelog-producer 这个选项：

none: 只保留 snapshot 级的变更日志，会丢失一部分中间变化。虽然 flink 本身也会辅助生成 changelog 但是开销太大不推荐。可以通过 `scan.remove-normalize=true` 关闭 flink 自身的补偿日志。

input: 保留上游输入的变更日志，一般上游输入为 CDC 时使用。

lookup: 在上游没有输入 CDC 日志时，依据 paimon 对数据的对比生成新的 changelog 供下游使用，在上游非 cdc 任务时，算是最准确的 changelog。

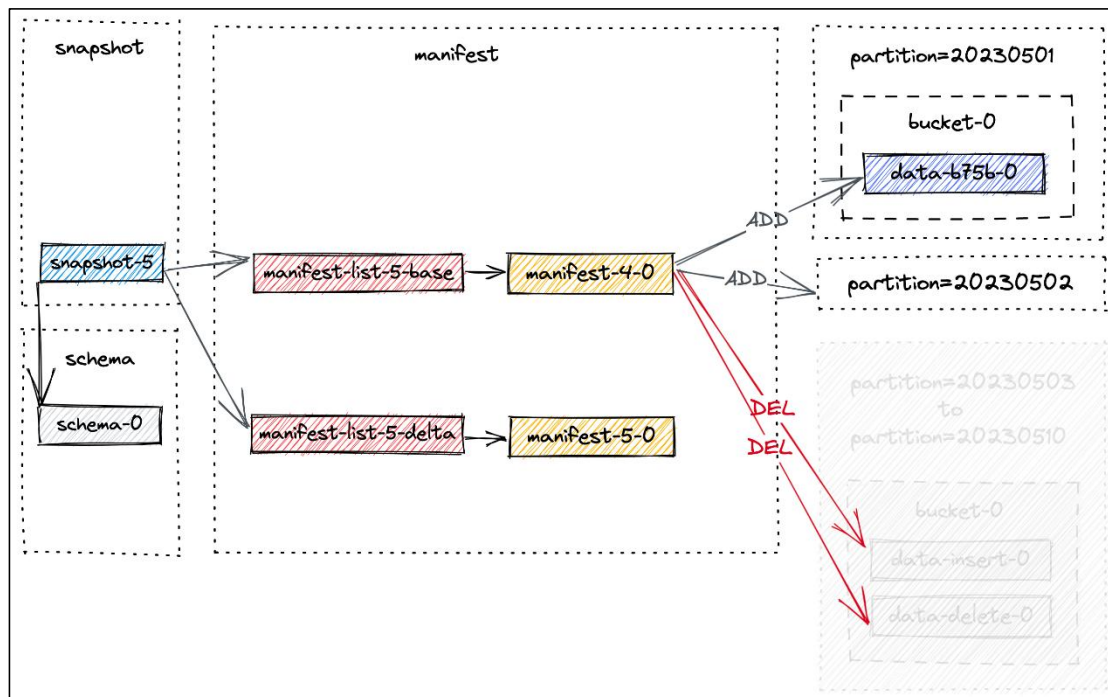
full-compaction: 相比 lookup 生成 changelog 间隔较大，可以通过设置 full compaction 间隔，在 full compaction 时产生最新的 changelog 给下游，性能相对 lookup 较好，但产生延迟受 full compaction 影响。如果设置了该类型的 changelog-producer，则 `full-compaction.delta-commits` 在无默认值自动变为 1。

6.8 快照过期

如果数据发生了删除，替换，不管是 Compaction 还是 Full Compaction，都不会删除任何数据文件。因为只要是历史快照还存在数据就有被“穿越”查询的可能性，所以不能删除。

但是如果历史数据真的是冗余，可以通过保留快照的数量来控制数据。

当快照数量小于 `snapshot.num-retained.min`（默认 10）时，没有快照会过期（即使满足 `snapshot.time-retained`（默认 1h）条件），之后 `snapshot.num-retained.max` 和 `snapshot.time-retained` 将被过期用于控制快照过期，直到剩余快照满足条件。



第 7 章 整合 Hive

数据湖的作为湖仓一体的一个重要特性就是可以完美支持基于 Hive 的数仓开发。

那么已经入湖的数据表结构和数据是否可以在 Hive 中无感查询呢？这一点 Paimon 较其他的数据湖产品支持的更好。

7.1 扩展 Hive

创建 hive 扩展目录

```
mkdir /opt/module/hive/auxlib
```

把 jar 包拷贝入扩展目录

```
cp paimon-hive-connector-3.1-0.8-20240503.002102-76.jar /opt/module/hive/auxlib
```

7.2 查看数据库和表

查看 hive 数据库。

```
hive(default)> show databases;
OK
database_name
default
gmall
gmall_lake2024
Time taken: 0.048 seconds, Fetched: 5 row(s)
```

查看数据表。

```
hive (gmall_lake2024)> use gmall_lake2024;
```

更多 [Java](#) - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网


```
OK
Time taken: 0.041 seconds
hive (gmall_lake2024)> show tables;
OK
tab_name
ods_base_category3_cdc
ods_base_dic_cdc
ods_base_province_cdc
ods_base_trademark_cdc
ods_order_detail_cdc
ods_order_info_cdc
ods_sku_info_cdc
ods_user_info_cdc
Time taken: 0.031 seconds, Fetched: 9 row(s)
hive (gmall_lake2024)>
```

查看表结构。

```
hive (gmall_lake2024)> show create table ods_order_detail_cdc;
OK
createtab_stmt
CREATE TABLE `ods_order_detail_cdc` (
  `id` bigint COMMENT '编号',
  `order_id` bigint COMMENT '订单编号',
  `sku_id` bigint COMMENT 'sku_id',
  `sku_name` varchar(200) COMMENT 'sku 名称 (冗余)',
  `img_url` varchar(200) COMMENT '图片名称 (冗余)',
  `order_price` decimal(10,2) COMMENT '购买价格 (下单时 sku 价格)',
  `sku_num` bigint COMMENT '购买个数',
  `create_time` timestamp COMMENT '创建时间',
  `source_type` varchar(20) COMMENT '来源类型',
  `source_id` bigint COMMENT '来源编号',
  `split_total_amount` decimal(16,2) COMMENT '',
  `split_activity_amount` decimal(16,2) COMMENT '',
  `split_coupon_amount` decimal(16,2) COMMENT '',
  `operate_time` timestamp COMMENT '')
COMMENT '订单明细表'
PARTITIONED BY (
  `dt` string)
ROW FORMAT SERDE
  'org.apache.paimon.hive.PaimonSerDe'
STORED BY
  'org.apache.paimon.hive.PaimonStorageHandler'
LOCATION

'hdfs://hadoop102:8020/paimon/hive/gmall_lake2024.db/ods_order_de
tail_cdc'
TBLPROPERTIES (
  'table_type'='PAIMON',
  'transient_lastDdlTime'='1715138681')
```

这里可以看到原来 paimon 中的表结构已经映射为 hive 表，而且分区也可以识别。

查看数据：

```
hive (gmall_lake2024)> select * from ods_base_province_cdc order by
id limit 10;
```



```
ods_base_province_cdc.id      ods_base_province_cdc.name      ods_base_province_cdc.region_id ods_base_province_cdc.
area_code      ods_base_province_cdc.iso_code ods_base_province_cdc.iso_3166_2 ods_base_province_cdc.create_t
ime      ods_base_province_cdc.operate_time
1      北京      1      110000      CN-11      CN-BJ      2021-12-14 00:00:00      NULL
2      天津      1      120000      CN-12      CN-TJ      2021-12-14 00:00:00      NULL
3      山西      1      140000      CN-14      CN-SX      2021-12-14 00:00:00      NULL
4      内蒙古      1      150000      CN-15      CN-NM      2021-12-14 00:00:00      NULL
5      河北      1      130000      CN-13      CN-HE      2021-12-14 00:00:00      NULL
6      上海      2      310000      CN-31      CN-SH      2021-12-14 00:00:00      NULL
7      江苏      2      320000      CN-32      CN-JS      2021-12-14 00:00:00      NULL
8      浙江      2      330000      CN-33      CN-ZJ      2021-12-14 00:00:00      NULL
9      安徽      2      340000      CN-34      CN-AH      2021-12-14 00:00:00      NULL
10     福建      2      350000      CN-35      CN-FJ      2021-12-14 00:00:00      NULL
Time taken: 41.54 seconds, Fetched: 10 row(s)
```

7.3 中文乱码问题的解决

基于 hive on spark 的 hive 计算，如果计算结果出现中文乱码的情况。请在 hive 中的 spark-defaults.conf 中增加：

```
spark.driver.extraJavaOptions=-Dfile.encoding=UTF-8
spark.executor.extraJavaOptions=-Dfile.encoding=UTF-8
```

第 8 章 kafka 日志数据入湖

8.1 环境准备

拷贝 flink-connector-kafka-3.1.0-1.17.jar 和 kafka-clients-3.3.1.jar 到 flink 的 lib 目录下。

注意 kafka-clients 要对应当前 kafka 的版本。

8.2 创建 Paimon 表

```
CREATE TABLE IF NOT EXISTS ods_log
(
  `common` string,
  `page` string,
  `displays` string,
  `actions` string,
  `start` string,
  `err` string,
  `ts` bigint,
  `dt` varchar(10)
)PARTITIONED BY (dt)
WITH (
  'bucket' = '2',
  'path' =
'hdfs://hadoop102:8020/paimon/hive/gmall_lake2024.db/ods_log',
  'sink.parallelism' = '2',
  'metastore.partitioned-table'='true'
);
```

8.3 创建 Kafka 映射表

```
CREATE temporary TABLE kafka_log (
  `common` string,
  `page` string,
  `displays` string,
  `actions` string,
  `start` string,
```

```
`err` string,
`ts` bigint
) WITH (
  'connector' = 'kafka',
  'topic' = 'topic_log',
  'properties.bootstrap.servers' = 'hadoop102:9092',
  'scan.startup.mode' = 'latest-offset',
  'properties.group.id' = 'ods_log',
  'format' = 'json'
);
```

注意 kafka 不是 paimon 的 catalog 中的固定表，所以必须要加 temporary。

8.4 装载数据

```
insert into ods_log
select `common`,
      `page`,
      `displays`,
      `actions`,
      `start`,
      `err`,
      `ts`,
      DATE_FORMAT(
        TO_TIMESTAMP_LTZ(`ts`, 3),
        'yyyy-MM-dd'
      )
from kafka_log;
```

这里把 log 日志中的 ts 的值转换为时间戳再转为 yyyy-MM-dd 的日期格式。

执行完成后可以通过 java -jar gmallxxxx.jar 生成电商日志数据。

8.5 查询结果

```
Flink SQL> select * from ods_log
```

op	common	page	displays	actions	start	err	ts	dt	action_id
I	{ "ar": "10", "ba": "Redmi", "ch": "notice", "loading_time": 1715216918266 }	<NULL>	<NULL>	<NULL>	<NULL>	<NULL>	1715216918266	2024-05-09	<NULL>
I	{ "ar": "3", "ba": "OPPO", "ch": "icon", "loading_time": 1715225656673 }	<NULL>	<NULL>	<NULL>	<NULL>	<NULL>	1715225656673	2024-05-09	<NULL>
I	{ "ar": "21", "ba": "iPhone", "ch": "install", "loading_time": 1715252592799 }	<NULL>	<NULL>	<NULL>	<NULL>	<NULL>	1715252592799	2024-05-09	<NULL>
I	{ "ar": "14", "ba": "OPPO", "ch": "icon", "loading_time": 1715226084667 }	<NULL>	<NULL>	<NULL>	<NULL>	<NULL>	1715226084667	2024-05-09	<NULL>
I	{ "ar": "20", "ba": "vivo", "ch": "icon", "loading_time": 1715263081409 }	<NULL>	<NULL>	<NULL>	<NULL>	<NULL>	1715263081409	2024-05-09	<NULL>
I	{ "ar": "4", "ba": "vivo", "ch": "install", "loading_time": 1715260600881 }	<NULL>	<NULL>	<NULL>	<NULL>	<NULL>	1715260600881	2024-05-09	<NULL>
I	{ "ar": "3", "ba": "OPPO", "ch": "install", "loading_time": 1715225666911 }	<NULL>	<NULL>	<NULL>	<NULL>	<NULL>	1715225666911	2024-05-09	<NULL>
I	{ "ar": "5", "ba": "vivo", "ch": "install", "loading_time": 1715230324798 }	<NULL>	<NULL>	<NULL>	<NULL>	<NULL>	1715230324798	2024-05-09	<NULL>
I	{ "ar": "14", "ba": "realme", "ch": "install", "loading_time": 1715254466359 }	<NULL>	<NULL>	<NULL>	<NULL>	<NULL>	1715254466359	2024-05-09	<NULL>
I	{ "ar": "27", "ba": "vivo", "ch": "install", "loading_time": 1715238669089 }	<NULL>	<NULL>	<NULL>	<NULL>	<NULL>	1715238669089	2024-05-09	<NULL>

第 9 章 DIM 层业务开发

9.1 开发要点

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网

利用 Tag，实现增量表转换为 Hive 中的每日全量表。

9.2 用户信息表

9.2.1 数据湖建表

```
CREATE TABLE `hive_catalog`.`gmall_lake2024`.`dim_user_info` (  
  `id` BIGINT NOT NULL,  
  `login_name` VARCHAR(200),  
  `nick_name` VARCHAR(200),  
  `user_level` VARCHAR(200),  
  `birthday` DATE,  
  `gender` VARCHAR(1),  
  `create_time` TIMESTAMP,  
  `operate_time` TIMESTAMP,  
  `status` VARCHAR(200),  
  CONSTRAINT `PK_id` PRIMARY KEY (`id`) NOT ENFORCED  
) WITH (  
  'path' =  
'hdfs://hadoop102:8020/paimon/hive/gmall_lake2024.db/dim_user_info',  
  'bucket' = '2',  
  'changelog-producer' = 'input',  
  'sink.parallelism' = '2',  
  'tag.automatic-creation' = 'process-time',  
  'tag.creation-period' = 'daily',  
  'tag.creation-delay' = '1 m',  
  'tag.num-retained-max' = '30',  
  'metastore.tag-to-partition'='dt',  
  'metastore.tag-to-partition.preview'='process-time',  
  'tag.num-retained-max'='365'  
)
```

关于快照

Paimon 的快照虽然可以查询历史数据，但在大多数情况下，一个作业会生成太多快照，并且表会根据表配置使旧快照过期。快照过期也会删除旧的数据文件，过期快照的历史数据无法再查询。

为了解决这个问题，可以基于快照创建标签。该标签将保留快照的 manifest 和数据文件。典型的用法是每天创建标签，然后可以保留每天的历史数据进行批量读取。

而且更加方便的是可以把快照的标签映射为 hive 的分区字段，这样即使不是每日全量表，也可以当成每日全量表在 hive 中使用。

参数解释：

参数	描述
'tag.automatic-creation' = 'process-time'	这里按照系统处理时间生成 tag。
'tag.creation-period' = 'daily'	每天生成。其他选项：hourly , two-hours

'tag.creation-delay' = '10 m'	延迟 10 分钟,即每天 00:10 分开始生成标签
'tag.num-retained-max' = '30'	最多保留 30 个标签, 如果每天 1 个的话则保留 30 天
'metastore.tag-to-partition'='dt'	会把标签的提交日期的前一日映射为一个 dt 字段,这样就可以把标签作为一个 hive 分区表来使用。在 hive 中好像是一个每日全量表,但是实际上是利用标签进行时间旅行出来的一个增量表。目的就是大大节省了存储开销,这就好像把一个每日全量表变为了一个拉链表的容量。
'metastore.tag-to-partition.preview'='process-time'	把当前最新数据映射为当前日期的分区中。

9.2.2 装载数据

```
insert into dim_user_info
select  `id` ,
       `login_name` ,
       `nick_name` ,
       `user_level` ,
       `birthday` ,
       `gender` ,
       `create_time` ,
       `operate_time` ,
       `status`
from `gmall_lake2024`.`ods_user_info_cdc`;
```

9.2.3 在 hive 中观察数据和分区

执行

```
hive (gmall_lake2024)> select dt,count(*)  from dim_user_info group
by dt;
```

会得到如下结果:

dt	_c1
2024-05-09	2549
2024-05-08	2549

说明:

执行这条语句的系统时间是 2024-05-09 日。dim_user_info 会自动产生两个分区。

2024-05-08 这个分区是因为，根据'tag.creation-period' = 'daily' 这个原则每天产生一个分区，而'tag.creation-delay' = '10 m'这个描述了在当日 00:10 会产生前一分区的分区。当日是 2024-05-09，那么就会产生 2024-05-08 的分区。

而 2024-05-09 这个分区是根据'metastore.tag-to-partition.preview'='process-time' 把当日最新数据映射出来的。这两个分区虽然数据量一直，好像是一份全量数据，但是底层实际上是一份数据通过 tag 映射出来的两个分区。

验证：

在 mysql 数据库中修改某一个用户的一条数据。

```
update user_info set gender='F' where id=4;
```

然后在 hive 中进行查询。

```
hive (gmall_lake2024)> select * from dim_user_info where id=4;
```

会发现在两个分区都查询出来该用户数据， 2024-05-08 的是旧数据，而 2024-05-09 的是最新数据。

这样在数仓的批处理模式下，所有维表完全可以作为一个每日全量表使用，而又不会存在较大的冗余，而且还可以支持查询当日的最新数据。

9.3 地区维度表

9.3.1 建表语句

```
CREATE TABLE `hive_catalog`.`gmall_lake2024`.`dim_province`
(
  `id`          bigint,
  `province_name` VARCHAR(200),
  `area_code`   VARCHAR(200),
  `iso_code`    VARCHAR(200),
  `iso_3166_2`  VARCHAR(200),
  `create_time` TIMESTAMP ,
  `operate_time` TIMESTAMP ,
  CONSTRAINT `PK_id` PRIMARY KEY (`id`) NOT ENFORCED
) WITH (
  'path' =
  'hdfs://hadoop102:8020/paimon/hive/gmall_lake2024.db/dim_province',
  'bucket' = '2',
  'changelog-producer' = 'input',
  'sink.parallelism' = '2',
  'tag.automatic-creation' = 'process-time',
  'tag.creation-period' = 'daily',
  'tag.creation-delay' = '10 m',
  'tag.num-retained-max' = '30',
  'metastore.tag-to-partition'='dt',
```

```
'metastore.tag-to-partition.preview'='process-time'
);
```

9.3.2 装载语句

```
insert into dim_province
select  pc.`id` ,
        pc.`name` ,
        `area_code` ,
        `iso_code` ,
        `iso_3166_2` ,
        pc.`create_time` ,
        pc.`operate_time`
from `gmall_lake2024`.`ods_base_province_cdc` pc
```

9.4 商品表

9.4.1 建表语句

```
CREATE TABLE IF NOT EXISTS `dim_sku_info` (
  `id` bigint,
  `spu_id` bigint,
  `price` decimal(10, 2),
  `sku_name` varchar(200),
  `sku_desc` varchar(2000),
  `weight` decimal(10, 2),
  `tm_id` bigint,
  `tm_name` varchar(100),
  `category3_id` bigint,
  `category3_name` varchar(50),
  `is_sale` tinyint,
  `create_time` TIMESTAMP,
  `operate_time` TIMESTAMP,
  CONSTRAINT `PK_id` PRIMARY KEY (`id`) NOT ENFORCED
) WITH (
  'path' = 'hdfs://hadoop102:8020/paimon/hive/gmall_lake2024.db/dim_sku_info',
  'bucket' = '2',
  'changelog-producer' = 'input',
  'sink.parallelism' = '2',
  'tag.automatic-creation' = 'process-time',
  'tag.creation-period' = 'daily',
  'tag.creation-delay' = '10 m',
  'tag.num-retained-max' = '30',
  'metastore.tag-to-partition'='dt',
  'metastore.tag-to-partition.preview'='process-time'
)
```

9.4.2 装载语句

```
set 'table.exec.sink.upsert-materialize'='NONE';
insert into `dim_sku_info`
select
  si.`id` ,
  si.`spu_id` ,
  si.`price` ,
  si.`sku_name` ,
```



```

si.`sku_desc` ,
si.`weight` ,
si.`tm_id` ,
t.tm_name as `tm_name` ,
si.`category3_id` ,
c3.name as `category3_name` ,
si.`is_sale` ,
si.`create_time` ,
si.`operate_time`
from ods_sku_info_cdc si
left join ods_base_category3_cdc c3 on c3.id = si.category3_id
left join ods_base_trademark_cdc t on t.id=si.tm_id

```

处理报错:

```

[ERROR] Could not execute SQL statement. Reason:
java.lang.IllegalArgumentException: Sink materializer must not be
used with Paimon sink. Please set
'table.exec.sink.upsert-materialize' to 'NONE' in Flink's config.

```

执行语句前, 增加 `set 'table.exec.sink.upsert-materialize'='NONE';`

这里注意执行涉及回撤操作 `upsert` 语句时, 要关闭 `flink` 的 `upsert-materializer`, 该配置主要用于是 `flink` 优化 `upsert` 回撤动作中的一些重复操作, 但是目前 `paimon`, 不支持该优化, 需要保持原始状态。

第 10 章 DWD 层业务开发

10.1 下单业务表(Flink SQL 方式处理)

10.1.1 处理重点

➤ 事实表多流 join。

10.1.2 建表语句

```

CREATE TABLE IF NOT EXISTS dwd_trade_order_detail
(
    `detail_id`          BIGINT not null,
    `order_id`           BIGINT,
    `user_id`            BIGINT,
    `sku_id`             BIGINT,
    `province_id`        BIGINT,
    `create_time`        TIMESTAMP,
    `operate_time`       TIMESTAMP,
    `sku_num`            BIGINT,
    `order_price`        DECIMAL(16, 2),
    `split_activity_amount` DECIMAL(16, 2),
    `split_coupon_amount` DECIMAL(16, 2),
    `split_total_amount` DECIMAL(16, 2),
    `dt`                 varchar(20),
    CONSTRAINT `PK_id` PRIMARY KEY (`detail_id`,`dt`) NOT ENFORCED
) PARTITIONED BY (dt)
WITH (

```

```
'bucket' = '2',
'path'
'hdfs://hadoop102:8020/paimon/hive/gmall_lake2024.db/dwd_trade_order_detail',
'sink.parallelism' = '2',
'changelog-producer' = 'input',
'metastore.partitioned-table'='true'
);
```

10.1.3 数据装载

```
set 'table.exec.sink.upsert-materialize'='NONE';
insert into dwd_trade_order_detail
select
    od.`id` ,
    od.`order_id` ,
    oi.`user_id` ,
    od.`sku_id` ,
    oi.`province_id` ,
    oi.`create_time` ,
    oi.`operate_time` ,
    od.`sku_num` ,
    od.`order_price` ,
    od.`split_activity_amount` ,
    od.`split_coupon_amount` ,
    od.`split_total_amount` ,
    oi.`dt`
from ods_order_info_cdc oi
inner join ods_order_detail_cdc od on oi.id=od.order_id
and oi.`dt`=od.`dt`
and order_status='1001';
```

这里的数据装载，主要就是两张事实表进行 join 操作。

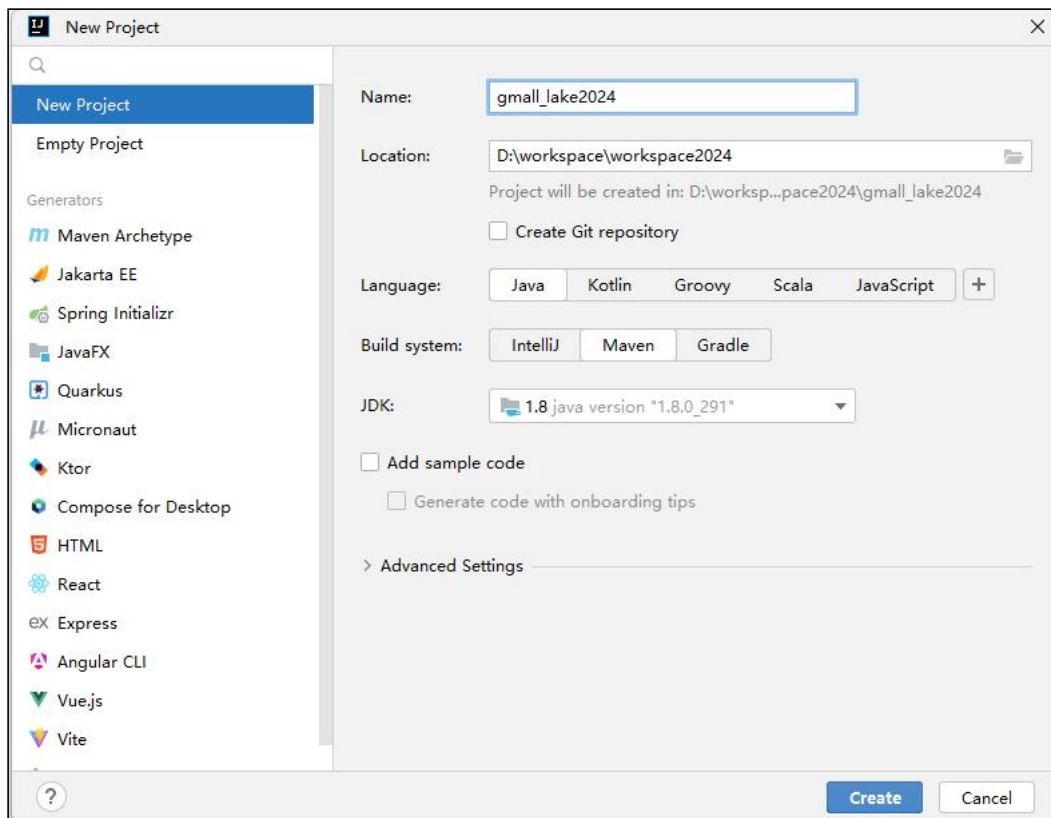
10.2 日志分流处理（Flink Api 方式处理）

10.2.1 处理重点

对于一些 sql 不方便做的操作，可以使用 Flink Api 处理 Paimon 中的数据。比如分流操作、自定义状态等。

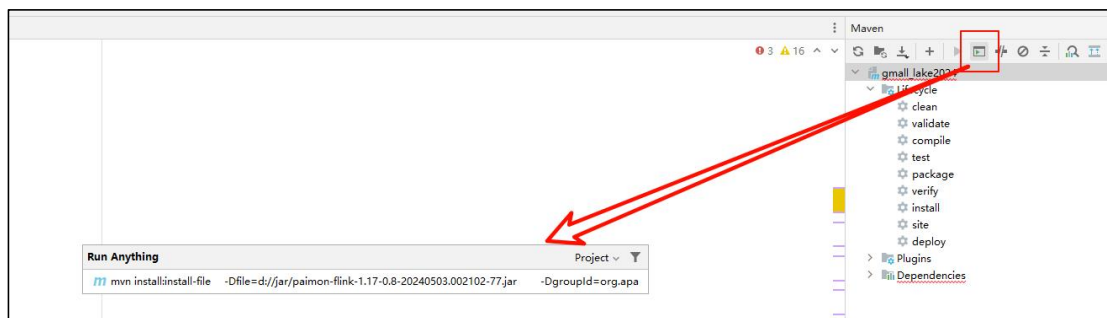
本案例以分流为例，利用 FlinkApi 将 Paimon 的 ods 层表 ods_log 分流为多个 dwd 表。

10.2.2 创建工程



10.2.3 安装依赖到本地库

因为本课程使用的是 paimon 的 0.8 版，目前 maven 仓库还没有可以直接下载的，需要手动安装 jar 到本地 maven 仓库。



在 idea 的 maven 命令窗口中，执行安装命令：

```
mvn install:install-file
-Dfile=d://jar/paimon-flink-1.17-0.8-20240503.002102-77.jar
-DgroupId=org.apache.paimon
-DartifactId=paimon-flink-1.17
-Dversion=0.8-SNAPSHOT
-Dpackaging=jar
```

注意修改 jar 的目录。

10.2.4 建立目标表

在 flink 客户端 gmall_lake2024 库下建立 paimon 表。

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

1) 页面访问表

```
CREATE TABLE IF NOT EXISTS dwd_traffic_page
(
    `common` string,
    `page` string,
    `displays` string,
    `actions` string,
    `dt` varchar(10)
) PARTITIONED BY (dt)
WITH (
    'bucket' = '2',
    'path' = 'hdfs://hadoop102:8020/paimon/hive/gmall_lake2024.db/dwd_traffic_page',
    'sink.parallelism' = '2',
    'metastore.partitioned-table'='true'
);
```

2) 启动表

```
CREATE TABLE IF NOT EXISTS dwd_traffic_start
(
    `common` string,
    `start` string,
    `dt` varchar(10)
) PARTITIONED BY (dt)
WITH (
    'bucket' = '2',
    'path' = 'hdfs://hadoop102:8020/paimon/hive/gmall_lake2024.db/dwd_traffic_start',
    'sink.parallelism' = '2',
    'metastore.partitioned-table'='true'
);
```

10.2.5 添加 pom.xml 中的依赖

pom.xml 文件

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>gmall_lake2024</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <java.version>1.8</java.version>
    </properties>

    <maven.compiler.source>${java.version}</maven.compiler.source>
```

```
<maven.compiler.target>${java.version}</maven.compiler.target>

<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
<
    <flink.version>1.17.2</flink.version>
    <flink-kafka.version>3.1.0-1.17</flink-kafka.version>
    <scala.version>2.12</scala.version>
    <hadoop.version>3.3.4</hadoop.version>
    <flink-cdc.version>2.4.0</flink-cdc.version>
    <fastjson.version>1.2.83</fastjson.version>
    <hbase.version>2.4.11</hbase.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.apache.flink</groupId>
        <artifactId>flink-streaming-java</artifactId>
        <version>${flink.version}</version>
        <scope>provided</scope>
    </dependency>

    <dependency>
        <groupId>org.apache.flink</groupId>
        <artifactId>flink-clients</artifactId>
        <version>${flink.version}</version>
        <scope>provided</scope>
    </dependency>

    <dependency><!--在 idea 运行的时候,可以打开 web 页面-->
        <groupId>org.apache.flink</groupId>
        <artifactId>flink-runtime-web</artifactId>
        <version>${flink.version}</version>
        <scope>provided</scope>
    </dependency>

    <dependency>
        <groupId>org.apache.flink</groupId>
        <artifactId>flink-json</artifactId>
        <version>${flink.version}</version>
        <scope>provided</scope>
    </dependency>

    <!--如果保存检查点到 hdfs 上, 需要引入此依赖-->
    <dependency>
        <groupId>org.apache.hadoop</groupId>
        <artifactId>hadoop-client</artifactId>
        <version>${hadoop.version}</version>
        <scope>provided</scope>
    </dependency>

    <dependency>
        <groupId>org.projectlombok</groupId>
```

```
<artifactId>lombok</artifactId>
<version>1.18.20</version>
<scope>provided</scope>
</dependency>

<!--Flink 默认使用的是 slf4j 记录日志, 相当于一个日志的接口, 我们这里使用 log4j 作为具体的日志实现-->
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>1.7.25</version>
  <scope>provided</scope>
</dependency>

<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-to-slf4j</artifactId>
  <version>2.14.0</version>
  <scope>provided</scope>
</dependency>

<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-table-api-java-bridge</artifactId>
  <version>${flink.version}</version>
  <scope>provided</scope>
</dependency>

<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-table-planner-loader</artifactId>
  <version>${flink.version}</version>
  <scope>provided</scope>
</dependency>

<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-table-runtime</artifactId>
  <version>${flink.version}</version>
  <scope>provided</scope>
</dependency>

<dependency>
  <groupId>org.apache.paimon</groupId>
  <artifactId>paimon-flink-1.17</artifactId>
  <version>0.8-SNAPSHOT</version>
</dependency>

<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-connector-base</artifactId>
  <version>${flink.version}</version>
</dependency>

<!--
https://mvnrepository.com/artifact/org.apache.hive/hive-metastore
```



```
-->
<dependency>
  <groupId>org.apache.hive</groupId>
  <artifactId>hive-metastore</artifactId>
  <version>3.1.3</version>
</dependency>

<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.0.33</version>
</dependency>
<dependency>
  <groupId>org.apache.hive</groupId>
  <artifactId>hive-metastore</artifactId>
  <version>3.1.2</version>
  <exclusions>
    <exclusion>
      <artifactId>log4j-slf4j-impl</artifactId>
      <groupId>org.apache.logging.log4j</groupId>
    </exclusion>
  </exclusions>
</dependency>
<dependency>
  <groupId>org.apache.hive</groupId>
  <artifactId>hive-common</artifactId>
  <version>3.1.2</version>
  <exclusions>

    <exclusion>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-log4j12</artifactId>
    </exclusion>
    <exclusion>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-simple</artifactId>
    </exclusion>
    <exclusion>
      <artifactId>log4j-slf4j-impl</artifactId>
      <groupId>org.apache.logging.log4j</groupId>
    </exclusion>
  </exclusions>
</dependency>

<dependency>
  <groupId>org.apache.hive</groupId>
  <artifactId>hive-exec</artifactId>
  <version>3.1.2</version>
  <exclusions>
    <exclusion>
      <artifactId>hadoop-annotations</artifactId>
      <groupId>org.apache.hadoop</groupId>
    </exclusion>
    <exclusion>

<artifactId>hadoop-yarn-server-common</artifactId>
  <groupId>org.apache.hadoop</groupId>
```

```
        </exclusion>
    </exclusion>

    <artifactId>hadoop-yarn-server-resourcemanager</artifactId>
        <groupId>org.apache.hadoop</groupId>
    </exclusion>
</exclusion>

    <artifactId>hadoop-yarn-server-applicationhistoryservice</artifactId>
        <groupId>org.apache.hadoop</groupId>
    </exclusion>
</exclusion>

    <artifactId>hadoop-yarn-server-web-proxy</artifactId>
        <groupId>org.apache.hadoop</groupId>
    </exclusion>
</exclusion>
    <artifactId>hadoop-yarn-common</artifactId>
        <groupId>org.apache.hadoop</groupId>
    </exclusion>
</exclusion>
    <artifactId>jackson-core-asl</artifactId>
        <groupId>org.codehaus.jackson</groupId>
    </exclusion>
</exclusion>
    <artifactId>jackson-mapper-asl</artifactId>
        <groupId>org.codehaus.jackson</groupId>
    </exclusion>
</exclusion>
    <artifactId>jersey-core</artifactId>
        <groupId>com.sun.jersey</groupId>
    </exclusion>
</exclusion>
    <artifactId>jersey-server</artifactId>
        <groupId>com.sun.jersey</groupId>
    </exclusion>
</exclusion>
    <artifactId>zookeeper</artifactId>
        <groupId>org.apache.zookeeper</groupId>
    </exclusion>

    <exclusion>
        <groupId>org.springframework</groupId>
        <artifactId>spring</artifactId>
    </exclusion>
    <exclusion>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-log4j12</artifactId>
    </exclusion>
    <exclusion>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-simple</artifactId>
    </exclusion>
    <exclusion>
        <artifactId>log4j-slf4j-impl</artifactId>
        <groupId>org.apache.logging.log4j</groupId>
```

```
        </exclusion>

    </exclusions>

</dependency>

<dependency>
    <groupId>com.alibaba</groupId>
    <artifactId>fastjson</artifactId>
    <version>${fastjson.version}</version>
</dependency>

<dependency>
    <groupId>org.apache.flink</groupId>
    <artifactId>flink-connector-files</artifactId>
    <version>${flink.version}</version>
    <scope>provided</scope>
</dependency>

<!--
https://mvnrepository.com/artifact/org.apache.flink/flink-statebackend-rocksdb -->
    <dependency>
        <groupId>org.apache.flink</groupId>
        <artifactId>flink-statebackend-rocksdb</artifactId>
        <version>${flink.version}</version>

    </dependency>

</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-shade-plugin</artifactId>
            <version>3.1.1</version>
            <executions>
                <execution>
                    <phase>package</phase>
                    <goals>
                        <goal>shade</goal>
                    </goals>
                    <configuration>
                        <artifactSet>
                            <excludes>

<exclude>com.google.code.findbugs:jsr305</exclude>
                                <exclude>org.slf4j:*</exclude>
                                <exclude>log4j:*</exclude>

<exclude>org.apache.hadoop:*</exclude>
                            </excludes>
                        </artifactSet>
                        <filters>
                            <filter>
                                <!-- Do not copy the signatures in the
```

```

META-INF folder. Otherwise, this might cause SecurityExceptions when
using the JAR. -->
        <!-- 打包时不复制 META-INF 下的签名文件,
避免报非法签名文件的 SecurityExceptions 异常-->
        <artifact>*:*</artifact>
        <excludes>
            <exclude>META-INF/*.SF</exclude>

<exclude>META-INF/*.DSA</exclude>

<exclude>META-INF/*.RSA</exclude>
        </excludes>
    </filter>
</filters>

    <transformers combine.children="append">
        <!-- The service transformer is needed to
merge META-INF/services files -->
        <!-- connector 和 format 依赖的工厂类打包时会
相互覆盖, 需要使用 ServicesResourceTransformer 解决-->
        <transformer

implementation="org.apache.maven.plugins.shade.resource.ServicesR
esourceTransformer"/>
    </transformers>
</configuration>
</execution>
</executions>
</plugin>
</plugins>
</build>
</project>

```

10.2.6 代码开发思路

- (1) 创建执行环境, 包括 checkpoint 参数
- (2) 加载 paimon 元数据目录
- (3) 读取 paimon 中的 ods 层表 ods_log 形成数据流
- (4) 对根据数据内容打标签, 并进行分流
- (5) 不同的流写入对应的数据表

参考官方 FlinkApi 样例: <https://paimon.apache.org/docs/0.8/program-api/flink-api/>

10.2.7 代码及说明

```

package com.atguigu.gmall.lake.dwd;

import com.alibaba.fastjson.JSON;
import org.apache.flink.api.common.typeinfo.Types;
import
org.apache.flink.contrib.streaming.state.EmbeddedRocksDBStateBack

```

```
end;
import org.apache.flink.streaming.api.CheckpointingMode;
import org.apache.flink.streaming.api.datastream.DataStream;
import
org.apache.flink.streaming.api.datastream.SingleOutputStreamOpera
tor;
import org.apache.flink.streaming.api.environment.CheckpointConfig;
import
org.apache.flink.streaming.api.environment.StreamExecutionEnviron
ment;
import org.apache.flink.streaming.api.functions.ProcessFunction;
import org.apache.flink.table.api.DataTypes;
import org.apache.flink.table.types.DataType;
import org.apache.flink.types.Row;
import org.apache.flink.types.RowKind;
import org.apache.flink.util.Collector;
import org.apache.flink.util.OutputTag;
import org.apache.paimon.catalog.Catalog;
import org.apache.paimon.catalog.Identifier;
import org.apache.paimon.flink.FlinkCatalogFactory;
import org.apache.paimon.flink.sink.FlinkSinkBuilder;
import org.apache.paimon.flink.source.FlinkSourceBuilder;
import org.apache.paimon.options.Options;
import org.apache.paimon.table.Table;

public class DwdLogApp {

    public static void main(String[] args) throws Exception {

        //1) 创建执行环境
        StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment();
        env.setParallelism(2);
        System.setProperty("HADOOP_USER_NAME", "atguigu");
        // 持久化 checkpoint 的配置
        env.enableCheckpointing(5000,
CheckpointingMode.EXACTLY_ONCE);
        env.setStateBackend(new EmbeddedRocksDBStateBackend(true));
        CheckpointConfig checkpointConfig =
env.getCheckpointConfig();

        checkpointConfig.setCheckpointStorage("hdfs://hadoop102:8020/flin
k-checkpoints");
        checkpointConfig.setMinPauseBetweenCheckpoints(2000);
        checkpointConfig.setTolerableCheckpointFailureNumber(5);
        checkpointConfig.setCheckpointTimeout(60000);

        checkpointConfig.enableExternalizedCheckpoints(CheckpointConfig.E
xternalizedCheckpointCleanup.RETAIN_ON_CANCELLATION);

        // 2) 加载 paimon 元数据目录
        // CREATE CATALOG hive_catalog WITH (
        //     'type' = 'paimon',
        //     'metastore' = 'hive',
        //     'uri' = 'thrift://hadoop102:9083',
        //     'hive-conf-dir' = '/opt/module/hive/conf',
```

```
//          'warehouse' = 'hdfs://hadoop102:8020/paimon/hive'
//);

//2.1 设定目录配置
Options catalogOptions= new Options();
catalogOptions.set("warehouse",
"hdfs://hadoop102:8020/paimon/hive");
catalogOptions.set("type", "paimon");
catalogOptions.set("'uri'", "'thrift://hadoop102:9083'");
catalogOptions.set("hive-conf-dir", "/opt/module/hive/conf");
catalogOptions.set("metastore", "hive");
//加载目录
Catalog catalog =
FlinkCatalogFactory.createPaimonCatalog(catalogOptions);
// 3) 读取 paimon 中的 ods 层表 ods_log 形成数据流
// 3.1 定义数据源表
Table sourceTable =
catalog.getTable(Identifier.create("gmall_lake2024", "ods_log"));
// 3.2 创键构造器
FlinkSourceBuilder sourceBuilder = new
FlinkSourceBuilder(sourceTable).env(env);
// 3.3 使用构造器创建数据流
DataStream<Row> dataStream = sourceBuilder.buildForRow();

//3.4 打印数据流
//
dataStream.executeAndCollect().forEachRemaining(System.out::print
ln);

// 4 对根据数据内容打标签, 并进行分流
//4.1 创建分流标签 可以设置多个...
OutputTag<Row> startTag = new OutputTag<Row>("startTag") {};

//4.2 解析数据结构, 根据数据打标签
SingleOutputStreamOperator<Row> pageDataStream =
dataStream.process(new ProcessFunction<Row, Row>() {
    @Override
    public void processElement(Row row, ProcessFunction<Row,
Row>.Context context, Collector<Row> collector) throws Exception {
        String common = row.getFieldAs("common");
        String page = row.getFieldAs("page");
        String displays = row.getFieldAs("displays");
        String actions = row.getFieldAs("actions");
        String start = row.getFieldAs("start");
        String err = row.getFieldAs("err");
        String dt = row.getFieldAs("dt");
        // page 流
        if (page != null) {
            Row pageRow = Row.of(common, page, displays,
actions, dt);
            collector.collect(pageRow);
        }
        // start 流
        else if (start != null) {
            Row startRow = Row.of(common, start, dt);
            context.output(startTag, startRow );
        }
    }
});
```

```

    }
    });
    //4.3 根据标签进行分流
    DataStream<Row> startDataStream=
pageDataStream.getSideOutput(startTag);
    //4.4 打印
    // pageDataStream.print("page:::");

    //5 不同的流写入对应的数据表
    //5.1 把数据流映射为表
    Table sinkPageTable =
catalog.getTable(Identifier.create("gmall_lake2024",
"dwd_traffic_page"));
    Table sinkStartTable =
catalog.getTable(Identifier.create("gmall_lake2024",
"dwd_traffic_start"));

    //5.2 定义不同的数据结构
    DataType pageSinkDataType =
        DataTypes.ROW(
            DataTypes.FIELD("common", DataTypes.STRING()),
            DataTypes.FIELD("page", DataTypes.STRING()),
            DataTypes.FIELD("displays",
DataTypes.STRING()),
            DataTypes.FIELD("actions", DataTypes.STRING()),
            DataTypes.FIELD("dt", DataTypes.STRING())
        );
    DataType startSinkDataType =
        DataTypes.ROW(
            DataTypes.FIELD("common", DataTypes.STRING()),
            DataTypes.FIELD("start", DataTypes.STRING()),
            DataTypes.FIELD("dt", DataTypes.STRING())
        );

    //5.3 定义不同的数据结构、表、流创建为构造器
    FlinkSinkBuilder sinkPageBuilder = new
FlinkSinkBuilder(sinkPageTable).forRow(pageDataStream,
pageSinkDataType);
    FlinkSinkBuilder sinkStartBuilder = new
FlinkSinkBuilder(sinkStartTable).forRow(startDataStream,
startSinkDataType);

    //5.4 构造器进行创建 sink
    sinkPageBuilder.build();
    sinkStartBuilder.build();

    //最终执行
    env.execute();
}
}

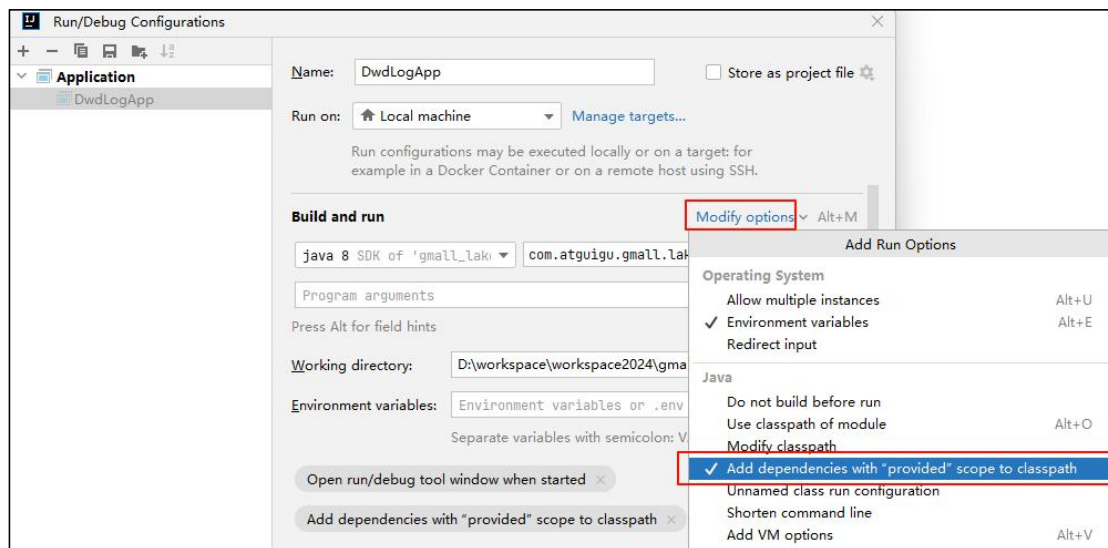
```

10.2.8 执行及注意事项

1) 如果用 idea 执行，需要把一份 hive-site.xml 拷贝到 d:\opt\module\hive\conf\目录下。盘符一般是代码工程所在的盘符。

更多 Java -大数据 -前端 -python 人工智能资料下载，可百度访问：尚硅谷官网

2) 执行 DwdLogApp 主程序，注意执行时要加载 provided。



10.2.9 观察数据

可以在 flinksql 或者 hive 中查询目标表数据是否到达。

```
hive (gmall_lake2024)> select * from dwd_traffic_page limit 2;
OK
dwd_traffic_page.common dwd_traffic_page.page dwd_traffic_page.displays dwd_traffic_page.actions dwd_traffic_page.dt
{"ar":"3","ba":"OPPO","ch":"xiaomi","is_new":"1","md":"OPPO Remo8","mid":"mid_355","os":"Android 13.0","sid":"6ab2398b-3c2a-4cd5-a081-710c3ac4067d","uid":"1413","vc":"v2.1.132"} {"during_time":10238,"page_id":"home","refer_id":"1"} [{"item":"1","item_type":"activity_id","pos_id":1,"pos_seq":1},{item":"2","item_type":"activity_id","pos_id":1,"pos_seq":1},{item":"3","item_type":"activity_id","pos_id":1,"pos_seq":1},{item":"4","item_type":"activity_id","pos_id":1,"pos_seq":1},{item":"8","item_type":"sku_id","pos_id":2,"pos_seq":0},{item":"19","item_type":"sku_id","pos_id":2,"pos_seq":1},{item":"9","item_type":"sku_id","pos_id":2,"pos_seq":2},{item":"26","item_type":"sku_id","pos_id":2,"pos_seq":3},{item":"19","item_type":"sku_id","pos_id":2,"pos_seq":4},{item":"29","item_type":"sku_id","pos_id":2,"pos_seq":5},{item":"29","item_type":"sku_id","pos_id":2,"pos_seq":6},{item":"26","item_type":"sku_id","pos_id":2,"pos_seq":7},{item":"9","item_type":"sku_id","pos_id":2,"pos_seq":8},{item":"26","item_type":"sku_id","pos_id":2,"pos_seq":9},{item":"30","item_type":"sku_id","pos_id":2,"pos_seq":10},{item":"26","item_type":"sku_id","pos_id":2,"pos_seq":11},{item":"28","item_type":"sku_id","pos_id":2,"pos_seq":12},{item":"33","item_type":"sku_id","pos_id":2,"pos_seq":13},{item":"1","item_type":"sku_id","pos_id":2,"pos_seq":14},{item":"12","item_type":"sku_id","pos_id":2,"pos_seq":15}] NULL 2024-05-09
{"ar":"5","ba":"vivo","ch":"xiaomi","is_new":"0","md":"vivo x90","mid":"mid_353","os":"Android 13.0","sid":"11245b20-6996-4c21-8786-ab5a1251a579","uid":"1987","vc":"v2.1.134"} {"during_time":5563,"page_id":"home","refer_id":"1"} [{"item":"1","item_type":"activity_id","pos_id":1,"pos_seq":1},{item":"2","item_type":"activity_id","pos_id":1,"pos_seq":1},{item":"3","item_type":"activity_id","pos_id":1,"pos_seq":1},{item":"4","item_type":"activity_id","pos_id":1,"pos_seq":1},{item":"23","item_type":"sku_id","pos_id":1,"pos_seq":4},{item":"28","item_type":"sku_id","pos_id":1,"pos_seq":5},{item":"16","item_type":"sku_id","pos_id":1,"pos_seq":6},{item":"14","item_type":"sku_id","pos_id":1,"pos_seq":7},{item":"7","item_type":"sku_id","pos_id":2,"pos_seq":0},{item":"18","item_type":"sku_id","pos_id":2,"pos_seq":1}] NULL 2024-05-09
Time taken: 2.681 seconds, Fetched: 2 row(s)
```

第 11 章 DWS 层业务开发

11.1 处理重点

- 对数据进行轻度聚合。
- lookup-join。

11.2 8.2 建立聚合表

创建 dws 层表（交易域省份粒度订单最近 1 日汇总表）。

```
CREATE TABLE IF NOT EXISTS dws_trade_province_order_1d
(
  `dt` varchar(20),
  `time_point` varchar(20) not null,
  `time_interval` varchar(20) not null,
  `province_id` bigint,
```

```

        `province_name`          varchar(20),
        `area_code`              varchar(20),
        `iso_code`               varchar(20),
        `iso_3166_2`             varchar(20),
        `order_original_amount_1d` decimal(16, 2) ,
        `activity_reduce_amount_1d` decimal(16, 2) ,
        `coupon_reduce_amount_1d` decimal(16, 2) ,
        `order_total_amount_1d` decimal(16, 2) ,
        CONSTRAINT `PK_id` PRIMARY KEY
        (`dt`,`time_point`,`time_interval`,`province_id`,`province_name`,
        `area_code`,
        `iso_code`,`iso_3166_2` ) NOT ENFORCED
    )
PARTITIONED BY (`dt`)
with(
    'merge-engine' = 'aggregation',
    'fields.order_original_amount_1d.aggregate-function' = 'sum',
    'fields.activity_reduce_amount_1d.aggregate-function' = 'sum',
    'fields.coupon_reduce_amount_1d.aggregate-function' = 'sum',
    'fields.order_total_amount_1d.aggregate-function' = 'sum',
    'bucket' = '2',
    'path' =
    'hdfs://hadoop102:8020/paimon/hive/gmall_lake2024.db/dws_trade_province_order_1d',
    'sink.parallelism' = '2',
    'changelog-producer' = 'lookup',
    'metastore.partitioned-table'='true'
);

```

其中该表为聚合表，意味着当明细数据写入后，会根据参数中的聚合策略（sum, count, max 等）对度量字段进行计算。

每个不属于主键的字段都可以被赋予一个聚合函数，由 `fields.<field-name>.aggregate-function` 表属性指定，否则它将使用 `last_non_null_value` 聚合作为默认值。

这样我们在 flink 任务中就可以不再进行任何聚合，而直接写入明细即可，所有聚合操作依靠聚合表完成。

关于订单个数：

目前的聚合表的自动聚合操作类型里不包含 `distinct count`，没有办法自动进行去重后 count，所以如果想取类似订单个数，这种操作只能通过 flink 进行窗口聚合获得。

11.3 关于 lookup-join 后写入数据

写入 dws 表。

```

set 'table.exec.sink.upsert-materialize'='NONE';
insert into dws_trade_province_order_1d
    select od.dt ,date_format(od.`create_time`,`yyyy-MM-dd HH:mm`)
    as time_point,
        '1m' as time_interval,

```

```
        od.`province_id` ,
        pv.`province_name`,
        pv.`area_code` ,
        pv.`iso_code` ,
        pv.`iso_3166_2` ,
        od.skunum * od.order_price ,
        od.`split_activity_amount` ,
        od.`split_coupon_amount` ,
        od.`split_total_amount`
    from
    (select o.*,PROCTIME() as proc_time from dwd_trade_order_detail
    o) as od
    join dim_province FOR SYSTEM_TIME AS OF od.proc_time AS pv
    on od.province_id=pv.id;
```

这里涉及使用了 lookup-join 的方式，即事实数据与维度数据发生关联，这里事实数据一定是查询最新的维度数据。

lookup-join 时，事实表需要一个当前处理时间的伪列，所以这里用 proctime() 函数生成该伪列，然后再与维度列进行关联。

第 12 章 利用 Starrocks 加速查询

虽然 paimon 既可以用于流式处理，也可以用于批处理查询。但是做为即席查询的数据库目前和主流的 OLAP 数据库还有一定差距。

那么对于 dws 层的表，我们可以把数据同步入 Starrocks，但只保留近期数据，而长周期历史数据保留在 paimon。这样即保证近期热数据的查询效率，又保证了长期历史数据的低存储成本。这样实现了冷热数据的隔离。而 Starrocks 可以通过外表的形式直接读取 paimon 中的数据，这样冷热数据不但可以隔离，还可以进行全周期的合并查询。

12.1 安装 Starrocks



参考安装文档

尚硅谷大数据技术
之StarRocks3.2.4

12.2 导入 Starrocks

把 paimon 中的数据导入 Starrocks 有两种方法：

一种就是使用 Flink 把 Paimon 作为 Source,把 Starrocks 作为 sink 端，进行流式写入。这种方式时效性最高，但是也相对更消耗资源，毕竟每个 dws 层的表都要启动一个持续运行的 Flink 任务。

另一种就是使用 Starrocks 的外表能力和物化视图的能力。外表能力可以让 Starrocks 直接读取 Paimon 中的数据,物化视图的能力可以让 Paimon 中的数据周期性的同步至 Starrocks

中的物理表中。这样数据需求端可以直接查询 Starrocks，来提升查询速度。而且操作起来更加简便。

12.2.1 在 Starrocks 中建立 paimon 的 catalog

```
CREATE EXTERNAL CATALOG paimon_catalog
PROPERTIES
(
    "type" = "paimon",
    "paimon.catalog.type" = "hive",

    "paimon.catalog.warehouse" = "hdfs://hadoop102:8020/paimon/hive",
    "hive.metastore.uris" = "thrift://hadoop102:9083"
)
```

查看已经建立的 catalog

```
show databases from paimon_catalog;
```

查询 catalog 下的 paimon 外表

```
select dt, province_name, substring(time_point,12,2)
time_hour,sum(order_total_amount_1d)
from paimon_catalog.gmall_lake2024.dws_trade_province_order_1d
group by dt,province_name,substring(time_point,12,2)
having dt='2024-05-01'
```

注意查询耗时

```
mysql> select dt, province_name, substring(time_point,12,2) time_hour,sum(order_total_amount_1d)
-> from paimon_catalog.gmall_lake2024.dws_trade_province_order_1d
-> group by dt,province_name,substring(time_point,12,2)
-> having dt='2024-05-01';
+-----+-----+-----+-----+
| dt      | province_name | time_hour | sum(order_total_amount_1d) |
+-----+-----+-----+-----+
| 2024-05-01 | 北京          | 14        | 8448.00                    |
+-----+-----+-----+-----+
1 row in set (1.40 sec)
```

12.2.2 利用物化视图进行数据同步

在 Starrocks 中建立数据库

```
create database gmall_lake2024;
use gmall_lake2024;
```

建立物化视图

```
CREATE MATERIALIZED VIEW mv_dws_trade_province_order_1d_30dl
PARTITION BY str2date (dt,'%Y-%m-%d')
PROPERTIES(
    "partition_ttl_number" = "30"
)
DISTRIBUTED BY HASH(`province_id`,`time_point`)
REFRESH ASYNC EVERY (INTERVAL 1 minute)
as
select
    `dt`,
    `time_point`,
    `time_interval`,
    `province_id`,
```

```
`province_name` ,
`area_code` ,
`iso_code` ,
`iso_3166_2` ,
`order_original_amount_1d` ,
`activity_reduce_amount_1d` ,
`coupon_reduce_amount_1d` ,
`order_total_amount_1d`
from paimon_catalog.gmall_lake2024.dws_trade_province_order_1d
where str2date (dt,'%Y-%m-%d') = current_date()
```

语句说明：

参数	描述
CREATE MATERIALIZED VIEW mv_dws_trade_province_order_1d_30dl	物化视图一般以 mv_ 开头，结尾 _30dl，标识该表保存了最近 30 天数据。
PARTITION BY str2date (dt,'%Y-%m-%d')	标识如果超过 30 天的历史数据将被清理掉
"partition_ttl_number" = "30"	延迟 10 分钟,即每天 00:10 分开始生成标签
DISTRIBUTED BY HASH(province_id`,`time_point`)	标识分桶字段
REFRESH ASYNC EVERY(INTERVAL 1 minute)	标识了物化视图的同步策略， Async 标识为异步，因为被查询表是外表，目前外表不支持同步刷新，必须设为异步。 这里设为每分钟 1 次，生产上可以更根据数据量和时效要求进行调整。
where str2date (dt,'%Y-%m-%d') = current_date()	where 条件限定了每次需要刷新的数据范围。

查询物化视图的元数据

```
show materialized views \G ;
```



```
id: 14526
database_name: gmall_lake2024
name: mv_dws_trade_province_order_id_30dl
refresh_type: ASYNC
is_active: true
inactive_reason: NULL
partition_type: RANGE
task_id: 14528
task_name: mv-14526
last_refresh_start_time: 2024-05-21 14:59:55
last_refresh_finished_time: 2024-05-21 14:59:57
last_refresh_duration: 2.082
last_refresh_state: SUCCESS
last_refresh_force_refresh: false
last_refresh_start_partition:
last_refresh_end_partition:
last_refresh_base_refresh_partitions: {}
last_refresh_mv_refresh_partitions:
last_refresh_error_code: 0
last_refresh_error_message:
rows: 35
text: CREATE MATERIALIZED VIEW `mv_dws_trade_province_order_id_30dl` (`dt`, `time_point`, `time_interval`, `province_id`, `province_name`, `area_code`, `iso_code`, `iso_3166_2`, `order_original_amount_id`, `activity_reduce_amount_id`, `coupon_reduce_amount_id`, `order_total_amount_id`)
PARTITION BY (str2date(`dt`, '%Y-%m-%d'))
DISTRIBUTED BY HASH(`province_id`, `time_point`)
REFRESH ASYNC EVERY(INTERVAL 1 MINUTE)
PROPERTIES (
"replicated_storage" = "true",
"partition_fll_number" = "30",
"replication_num" = "1",
"storage_medium" = "HDD"
)
AS SELECT `dws_trade_province_order_id`.`dt`, `dws_trade_province_order_id`.`time_point`, `dws_trade_province_order_id`.`time_interval`, `dws_trade_province_order_id`.`province_id`, `dws_trade_province_order_id`.`province_name`, `dws_trade_province_order_id`.`area_code`, `dws_trade_province_order_id`.`iso_code`, `dws_trade_province_order_id`.`iso_3166_2`, `dws_trade_province_order_id`.`order_original_amount_id`, `dws_trade_province_order_id`.`activity_reduce_amount_id`, `dws_trade_province_order_id`.`coupon_reduce_amount_id`, `dws_trade_province_order_id`.`order_total_amount_id`
FROM `paimon_catalog`.`gmall_lake2024`.`dws_trade_province_order_id`
WHERE ((str2date(`dws_trade_province_order_id`.`dt`, '%Y-%m-%d')) <= (current_date())) AND ((str2date(`dws_trade_province_order_id`.`dt`, '%Y-%m-%d')) > (current_date() - INTERVAL 30 DAY));
```

注意观察是否查询成功以及查询耗时，如果耗时太长则考虑增加刷新闻隔。

默认情况下，建立视图后会立刻除非一次刷新。

12.3 利用物化视图查询数据库

使用物化视图目前有两种方式：

➤ 一种是正常查询原表，Starrocks 的优化器会自动匹配物化视图，如果有物化视图可用的结果优化器会改写执行计划，不进行实际计算而是读取物化视图预存的结果。这样的好处是把物化视图作为预计算加速器，不用调整已有的数据查询 SQL。

但是目前的版本，这种方式对于原表为外表的情况，并不好用。

➤ 另一种就是直接把物化视图作为一张实体表进行查询。

获取各个省市当日分时下单金额

```
select dt, province_name, substring(time_point,12,2)
time_hour,sum(order_total_amount_id)
from mv_dws_trade_province_order_id_30dl
group by dt,province_name,substring(time_point,12,2)
having dt='2024-05-01'
```

```
mysql> select dt, province_name, substring(time_point,12,2) time_hour,sum(order_total_amount_id)
-> from mv_dws_trade_province_order_id_30dl
-> group by dt,province_name,substring(time_point,12,2)
-> having dt='2024-05-01';
```

```
+-----+-----+-----+-----+
| dt      | province_name | time_hour | sum(order_total_amount_id) |
+-----+-----+-----+-----+
| 2024-05-01 | 北京          | 14        | 8448.00                     |
+-----+-----+-----+-----+
1 row in set (0.14 sec)
```

获取最近 365 日各省下单金额

```
select province_name, sum(order_total_amount_1d)
from
    (select * from mv_dws_trade_province_order_1d_30dl
     union all
     select * from paimon_catalog.gmall_lake2024.dws_trade_province_order_1d
     where str2date(dt,'%Y-%m-%d') > current_date() - interval 30
     day and str2date(dt,'%Y-%m-%d') < current_date() - interval 365 day
     ) order_all
group by province_name
```

这里利用了冷热数据的合并查询

```
mysql> select province_name, sum(order_total_amount_1d)
-> from
-> (select * from mv_dws_trade_province_order_1d_30dl
-> union all
-> select * from paimon_catalog.gmall_lake2024.dws_trade_province_order_1d
-> where str2date(dt,'%Y-%m-%d') > current_date() - interval 30 day and str2date(dt,'%Y-%m-%d') < current_date() - interval 365 day
-> ) order_all
-> group by province_name
-> ;
```

province_name	sum(order_total_amount_1d)
新疆	6029.10
广西	158976.10
湖南	14696.00
山西	2899.00
云南	291112.20
浙江	8197.00
贵州	8279.10
西藏	3398.00
天津	6029.10
上海	208640.10
江苏	149988.00
河南	176719.20
陕西	326323.30
吉林	9649.00
宁夏	11749.00
青海	8197.00
台湾	298869.20
北京	123431.00
辽宁	11.00
河北	106691.10

20 rows in set (1.30 sec)

第 13 章 利用 Doris 加速查询（参考，不做讲解）

虽然 paimon 既可以作为流式写入，也可以用于批处理查询。但是做为即席查询的数据库目前和主流的 OLAP 数据库还有一定差距。

对于 dws 层的表，我们把数据同步入 doris，但只保留近期数据，而长周期历史数据保留在 paimon。这样即保证近期热数据的查询效率，又保证了长期历史数据的低存储成本。

相比 Starrocks，Doris 目前还不支持 Paimon 外表，所以只能靠 Flink 实时任务写入。

13.1 安装 Doris



00_尚硅谷大数据
技术之Doris.docx

13.2 拷贝 Flink 的 Doris 连接器

flink-doris-connector-1.17-1.6.0.jar 包到 flink-1.17.2/lib 下。

13.3 Doris 创建数据库

登录 doris，并在 doris 创建数据库。

```
mysql> create database gmall_lake2024;  
Query OK, 0 rows affected (0.07 sec)
```

```
mysql> use gmall_lake2024;  
Database changed
```

13.4 Doris 中建立数据表

```
CREATE TABLE IF NOT EXISTS  
gmall_lake2024.dws_trade_province_order_1d  
(  
    `dt` date,  
    `time_point` varchar(20) ,  
    `time_interval` varchar(20) ,  
    `province_id` bigint,  
    `province_name` varchar(20),  
    `area_code` varchar(20),  
    `iso_code` varchar(20),  
    `iso_3166_2` varchar(20),  
    `order_original_amount_1d` decimal(16, 2) ,  
    `activity_reduce_amount_1d` decimal(16, 2) ,  
    `coupon_reduce_amount_1d` decimal(16, 2) ,  
    `order_total_amount_1d` decimal(16, 2)  
)  
ENGINE=olap  
UNIQUE  
KEY(`dt`,`time_point`,`time_interval`,`province_id`,`province_name`,`area_code`,`iso_code`,`iso_3166_2`)  
PARTITION BY RANGE(`dt`)(  
DISTRIBUTED BY HASH(`time_point`,`province_id`) BUCKETS 10  
PROPERTIES(  
    "dynamic_partition.enable" = "true",  
    "dynamic_partition.time_unit" = "DAY",  
    "dynamic_partition.start" = "-30",  
    "dynamic_partition.end" = "3",  
    "dynamic_partition.prefix" = "p",  
    "dynamic_partition.buckets" = "10",  
    "replication_num" = "1",  
    "dynamic_partition.create_history_partition" = "true",  
    "dynamic_partition.history_partition_num" = "30"  
);
```

由于聚合操作已经在 paimon 中完成，该表只是用于同步 paimon 中的聚合结果，所以 doris 的表不需要再进行自动聚合，而只选择 unique key 模式表。

这里为了降低 doris 存储压力，所以 doris 只保存短周期（30 天内）数据，超过该期限自动删除，通过"dynamic_partition.start" = "-30" 参数实现。

13.5 同步语句脚本

建立 flinksql 对 doris 的临时映射表。

```
CREATE TEMPORARY TABLE doris_dws_trade_province_order_1d (
  `dt` date,
  `time_point` varchar(20) not null,
  `time_interval` varchar(20) not null,
  `province_id` bigint,
  `province_name` varchar(20),
  `area_code` varchar(20),
  `iso_code` varchar(20),
  `iso_3166_2` varchar(20),
  `order_count_1d` bigint,
  `order_original_amount_1d` decimal(16, 2) ,
  `activity_reduce_amount_1d` decimal(16, 2) ,
  `coupon_reduce_amount_1d` decimal(16, 2) ,
  `order_total_amount_1d` decimal(16, 2)
)
WITH (
  'connector' = 'doris',
  'fenodes' = 'hadoop102:8030',
  'table.identifier' = 'gmall_lake2024.dws_trade_province_order_1d',
  'username' = 'root',
  'password' = '000000'
);
```

注意由于该表是 doris 在 flink 的映射表，而不是 paimon 的表不能保存在 paimon 的元数据中，所以只能建立为临时表（TEMPORARY TABLE）。

同步语句，在 flink-sql 控制台执行。

```
insert into doris_dws_trade_province_order_1d
select
  to_date(`dt`) ,
  `time_point` ,
  `time_interval` ,
  `province_id` ,
  `province_name` ,
  `area_code` ,
  `iso_code` ,
  `iso_3166_2` ,
  `order_original_amount_1d` ,
  `activity_reduce_amount_1d` ,
  `coupon_reduce_amount_1d` ,
  `order_total_amount_1d`
from dws_trade_province_order_1d
WHERE to_date(`dt`) BETWEEN CURRENT_DATE - INTERVAL '30' DAY AND
CURRENT_DATE;
```

由于 doris 的 dt 字段要用于动态分区,所以必须是 date 类型,所以该语句需要用 to_date() 进行一下转换。

同时由于 doris 中只保留最近的 30 天内数据所以这里增加了最近 30 天的过滤条件。

13.6 Doris 中查询

可以在 doris 中查询 dws 表实现指标查询。

```
mysql> select * from dws_trade_province_order_ld;
```

dt	time_point	time_interval	province_id	province_name	area_code	iso_code	iso_3166_2	order_count_ld
2024-04-30	2024-04-30 10:00	minute	1	北京	110000	CN-11	CN-BJ	12
2024-04-30	2024-04-30 14:00	minute	1	北京	110000	CN-11	CN-BJ	2
2024-04-29	2024-04-29 10:00	minute	1	北京	110000	CN-11	CN-BJ	10
2024-04-30	2024-04-30 13:00	minute	1	北京	110000	CN-11	CN-BJ	2
2024-04-30	2024-04-30 14:11	minute	1	北京	110000	CN-11	CN-BJ	2
2024-04-30	2024-04-30 11:00	minute	1	北京	110000	CN-11	CN-BJ	2
2024-04-30	2024-04-30 14:12	minute	1	北京	110000	CN-11	CN-BJ	3

当数据到达 doris 后,就可以以即时的方式查询当日最新数据。比如查询当日各省份分时下单金额。

```
select      province_name,      dt,      substring(time_point,12,2)
time_hour,sum(order_total_amount_ld)
from dws_trade_province_order_ld where dt ='2024-04-30' group by
province_name,dt ,substring(time_point,12,2)
order by province_name,time_hour
```

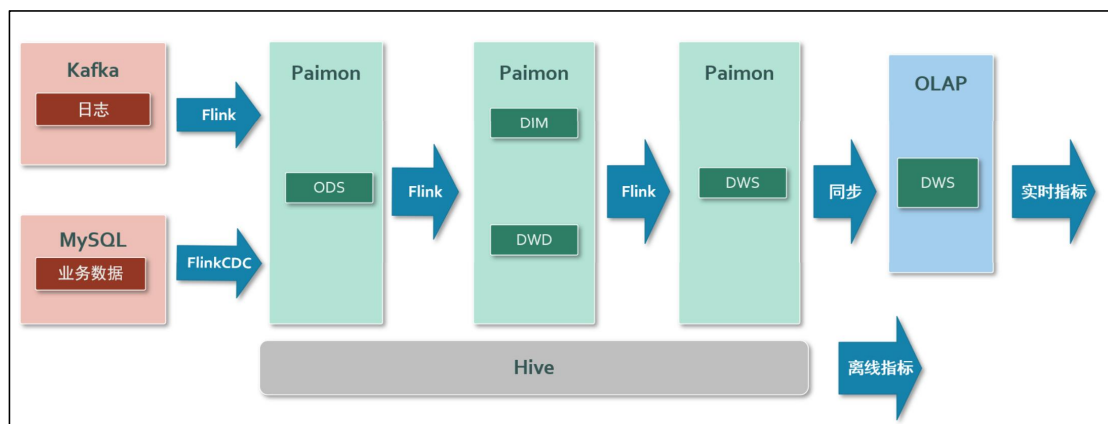
执行效果。

```
mysql> select * from dws_trade_province_order_ld;
```

dt	time_point	time_interval	province_id	province_name	area_code	iso_code	iso_3166_2	order_count_ld
2024-04-30	2024-04-30 10:00	minute	1	北京	110000	CN-11	CN-BJ	12
2024-04-30	2024-04-30 14:00	minute	1	北京	110000	CN-11	CN-BJ	2
2024-04-29	2024-04-29 10:00	minute	1	北京	110000	CN-11	CN-BJ	10
2024-04-30	2024-04-30 13:00	minute	1	北京	110000	CN-11	CN-BJ	2
2024-04-30	2024-04-30 14:11	minute	1	北京	110000	CN-11	CN-BJ	2
2024-04-30	2024-04-30 11:00	minute	1	北京	110000	CN-11	CN-BJ	2
2024-04-30	2024-04-30 14:12	minute	1	北京	110000	CN-11	CN-BJ	3

第 14 章 总结

通过一个完整的处理流程来理解，如何通过 Paimon+Flink+OLAP+Hive 实现一套流批一体的架构。



整个项目架构

1) 相比 Flink+Kafka 的流处理实时数仓：

(1) 替换掉了 kafka 作为中间表(ods,dim,dwd,dws)，利用 hdfs 进行存储，这样有更低的存储成本。

(2) 同时中间表(ods,dim,dwd,dws)随时都可以利用 hive 进行批处理的分析能力，甚至使用现有的 HiveSQL 脚本进行计算。

2) 相比纯 Hive 的批处理数仓：

- (1) 有更高的时效性（T+0 分钟级）
- (2) 对接 OLAP 后有更灵活快捷的分析能力
- (3) 对于维度的每日全量数据，通过 Tag 压缩为增量表的级别。

最重要的就是在结合了两种优势的情况下，只需要维护一套开发代码即可。